

B.TECH FIRST YEAR

ACADEMIC YEAR: 2025-2026

Course name: Digital System

Course code : ECE 1001

lecture series no : 01(one)

Credits : 3

Mode of delivery : online (Power point presentation)

Faculty :

Email-id :

PROPOSED DATE OF DELIVERY:

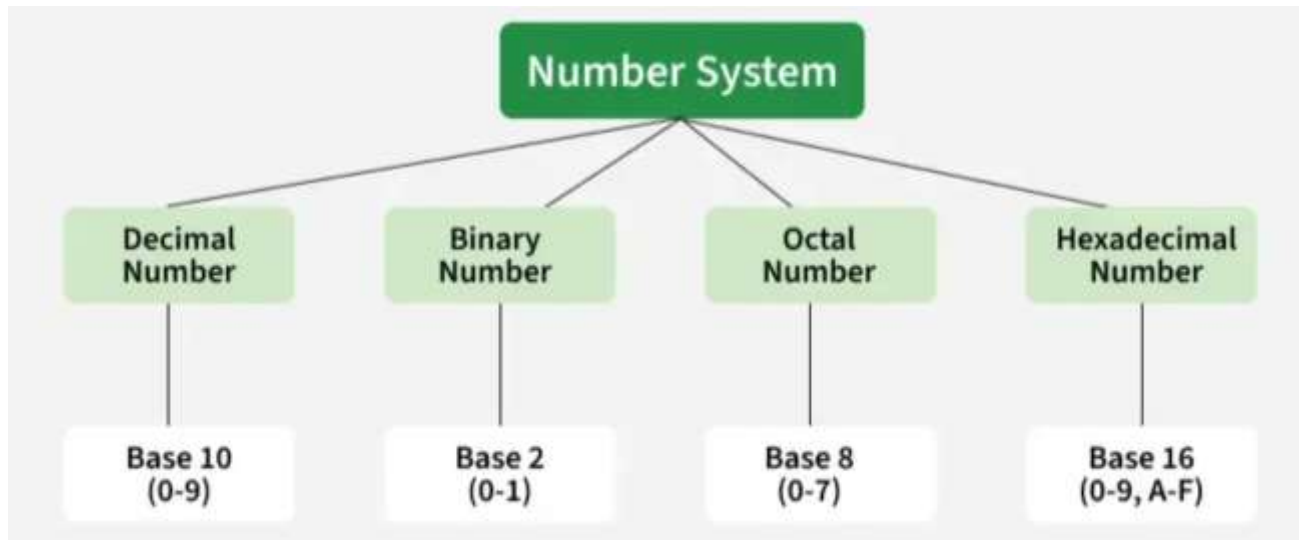


Tutorial:

Number Systems and Base Conversion

What is Number System ?

- A number system is a way to represent and express numbers using a consistent set of symbols or digits.
- A number system uses a base (or radix) to represent values.
- The base refers to the number of unique digits, including zero, that a system uses to represent numbers.
- Most commonly used number systems are Decimal (base-10), Binary (base-2), Octal (base-8), and Hexadecimal (base-16).
- Each system has its own set of rules for representing.
- Digital systems primarily rely on the Binary system for data processing.



Electronic and digital systems use various number systems such as Decimal, Binary, Hexadecimal and Octal, which are essential in computing

- Binary (base-2) is the foundation of digital systems.
- Hexadecimal (base-16) and Octal (base-8) are commonly used to simplify the representation of binary data.
- The Decimal system (base-10) is the standard system for everyday calculations.
- Other number systems like Duodecimal (base-12), are less commonly used but have specific applications in certain fields.

1. Decimal Number System

- The Decimal system is a base-10 number system.
- It uses ten digits: 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9.
- Each digit's place value is a power of 10 (e.g., 10^0 , 10^1 , 10^2).
- It is the standard system for everyday counting and calculations.

2. Binary Number System

- The Binary system is a base-2 number system.
- It uses two digits: 0 and 1.
- Each digit's place value is a power of 2 (e.g., 2^0 , 2^1 , 2^2).
- The Binary system is the foundation for data representation in computers and [digital electronics](#).

3. Octal Number System

- The Octal system is a base-8 number system.
- It uses eight digits: 0, 1, 2, 3, 4, 5, 6 and 7.
- Each digit's place value is a power of 8 (e.g., 8^0 , 8^1 , 8^2).
- It is often used to simplify the representation of binary numbers by grouping them into sets of three bits.

4. Hexadecimal Number System

- The Hexadecimal system is a base-16 number system.
- It uses sixteen digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E and F (where A = 10, B = 11, etc.).
- Each digit's place value is a power of 16 (e.g., 16^0 , 16^1 , 16^2).
- Hexadecimal simplifies binary by representing every 4 bits as one digit (0-F).

Number System Conversion Methods

A number N in base or radix b can be written as:

$$(N)_b = d_{n-1} d_{n-2} \dots d_1 d_0 . d_{-1} d_{-2} \dots d_{-m}$$

In the above, d_{n-1} to d_0 is the integer part, then follows a radix point and then d_{-1} to d_{-m} is the fractional part.

d_{n-1} = Most significant bit (MSB)

d_{-m} = Least significant bit (LSB)

Base	Representation
2	Binary
8	Octal
10	Decimal
16	Hexadecimal

1. Decimal to Binary Number System Conversion

For Integer Part:

- Divide the decimal number by 2.
- Record the remainder (0 or 1).
- Continue dividing the quotient by 2 until the quotient is 0.
- The binary equivalent is the remainders read from bottom to top.

Example: $(10.25)_{10}$

2	10	0
2	5	1
2	2	0
	1	

$(10)_{10} = (1010)_2$

Fractional Part

$0.25 \times 2 = 0.50$
 $0.50 \times 2 = 1.00$

$(0.25)_{10} = (0.01)_2$

For Fractional Part:

- Multiply the fractional part by 2.
- Record the integer part (0 or 1).
- Take the fractional part of the result and repeat the multiplication.
- Continue until the fractional part becomes 0 or reaches the desired precision.
- The binary equivalent is the integer parts recorded in sequence.

2. Binary to Decimal Number System Conversion

For Integer Part:

- Write down the binary number.
- Multiply each digit by 2 raised to the power of its position, starting from 0 (rightmost digit).
- Add up the results of these multiplications.
- The sum is the decimal equivalent of the binary integer

For Fractional Part:

- Write down the binary fraction.
- Multiply each digit by 2 raised to the negative power of its position, starting from -1 (first digit after the decimal point).
- Add up the results of these multiplications.
- The sum is the decimal equivalent of the binary fraction

Example: $(1010.01)_2$

$$1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 8 + 0 + 2 + 0 + 0 + 0.25 = 10.25$$

Thus, $(1010.01)_2 = (10.25)_{10}$

3. Decimal to Octal Number System Conversion

For Integer Part:

- Divide the decimal number by 8.
- Record the remainder (0 to 7).
- Continue dividing the quotient by 8 until the quotient is 0.
- The octal equivalent is the remainders read from bottom to top

Example: $(10.25)_{10}$

For Integer Part (10):

Divide 10 by 8 → Quotient = 1, Remainder = 2
Divide 1 by 8 → Quotient = 0, Remainder = 1

Octal equivalent = 12 (write the remainder, read from bottom to top). So, the octal equivalent of the integer part 10 is 12.

For Fractional Part:

- Multiply the fractional part by 8.
- Record the integer part (0 to 7).
- Take the fractional part of the result and repeat the multiplication.
- Continue until the fractional part becomes 0 or reaches the desired precision.
- The octal equivalent is the integer parts recorded in sequence.

For Fractional Part (0.25):

Multiply 0.25 by 8 → Result = 2.0,
Integer part = 2

The fractional part ends here as the result is now 0. So, the octal equivalent of the fractional part 0.25 is 0.2.

The octal equivalent of $(10.25)_{10} = (12.2)_8$

4. Octal to Decimal Number System Conversion

For Integer Part:

- Write down the octal number.
- Multiply each digit by 8 raised to the power of its position, starting from 0 (rightmost digit).
- Add up the results of these multiplications.
- The sum is the decimal equivalent of the octal integer.

For Fractional Part:

- Write down the octal fraction.
- Multiply each digit by 8 raised to the negative power of its position, starting from -1 (first digit after the decimal point).
- Add up the results of these multiplications.
- The sum is the decimal equivalent of the octal fraction.

Example: $(12.2)_8$

$$1 \times 8^1 + 2 \times 8^0 + 2 \times 8^{-1} = 8 + 2 + 0.25 = 10.25$$

Thus, $(12.2)_8 = (10.25)_{10}$



**MANIPAL UNIVERSITY
JAIPUR**

(University under Section 2(f) of the UGC Act)



B.TECH FIRST YEAR

ACADEMIC YEAR: 2025-2026



COURSE NAME: DIGITAL SYSTEM

COURSE CODE : ECE 1001

LECTURE SERIES NO : 01(ONE)

CREDITS : 3

MODE OF DELIVERY : ONLINE (POWER POINT PRESENTATION)

FACULTY :

EMAIL-ID :

PROPOSED DATE OF DELIVERY:



**MANIPAL UNIVERSITY
JAIPUR**

VISION

Global Leadership in Higher Education and Human Development

MISSION

- Be the most preferred University for innovative and interdisciplinary learning
- Foster academic, research and professional excellence in all domains
- Transform young minds into competent professionals with good human values

VALUES

Integrity, Transparency, Quality,
Team Work, Execution with Passion, Humane Touch



SESSION OUTCOME

"APPLY BOOLEAN ALGEBRA TO
SIMPLIFY LOGICAL EXPRESSIONS
AND IMPLEMENT LOGIC FUNCTIONS
USING BASIC AND UNIVERSAL
GATES "



ASSIGNMENT

QUIZ

MID TERM EXAMINATION –II

END TERM EXAMINATION

ASSESSMENT CRITERIA'S



PROGRAM OUTCOMES MAPPING WITH CO1

[PO1]

APPLY BOOLEAN ALGEBRA TO SIMPLIFY
LOGICAL EXPRESSIONS AND IMPLEMENT
LOGIC FUNCTIONS USING BASIC AND
UNIVERSAL GATES.

BOOLEAN ALGEBRA

Boolean algebra is the mathematical framework on which logic design based.

It is used in synthesis & analysis of binary logical function.

George Boole in 1854 invented a new kind of algebra known as Boolean algebra.

It is sometimes called switching algebra.

- $(A+B)(B+C)=AB+B.B+AC+BC$

- **$A+A'B=(A+A')(A+B)=A+B$**

- (A
- AND
- OR
- NOT (single input and single output)

- **$A+BC = (A+B)(A+C)$**

- **$A+A'B=(A+A')(A+B)$**

BASIC LAWS OF BOOLEAN ALGEBRA

- **Laws of complementation:** The term complement means invert. i.e. to change 0's to 1's and 1' to 0's.
- The following are the laws of complement. Here A is Boolean variables which can either value '0' or '1'.

- **“OR” laws (logical ‘OR’ operation)**

$$\begin{array}{llll} 0+0=0; & 0+1=1; & 1+0=1; & 1+1=1 \\ 1+A=1; & A+A' =1; & A+A=A; & 1+A' =1 \end{array}$$

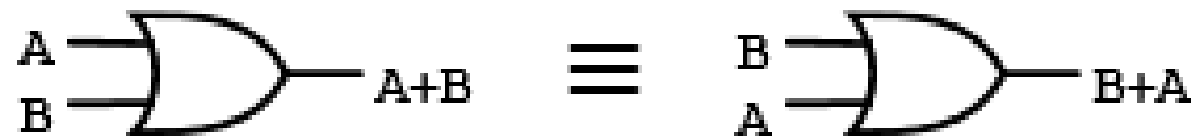
- **“AND” laws**

$$0.0=0; \quad 0.1=0; \quad 1.0=0; \quad 1.1=1; \quad A.A' =0; \quad A.A=A$$

COMMUTATIVE LAW:

- **Property 1:** This states that the order in which the variables OR makes no difference in output. i.e. $A+B=B+A$

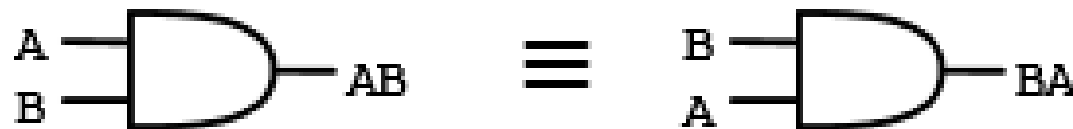
A	B	A+B		B	A	B+A
0	0	0		0	0	0
0	1	1	=	1	0	1
1	0	1		0	1	1
1	1	1		1	1	1



COMMUTATIVE LAW:

Property 2: This property of multiplication states that the order in which the variables are AND makes no difference in the output. i.e. $A.B=B.A$

A	B	A.B		B	A	B.A
0	0	0		0	0	0
0	1	0	=	1	0	0
1	0	0		0	1	0
1	1	1		1	1	1



ASSOCIATIVE PROPERTY

- **Property1:** This property states that in the OR'ing of the several variables, the result is same regardless of grouping of variables. For three variables i.e.(A OR'ed with B)or'ed with C is same as A OR'ed with (B OR'ed with C)
- i.e. $(A+B)+C = A+(B+C)$

A	B	C	A+B	B+C	(A+B)+C		A+(B+C)
0	0	0	0	0	0		0
0	0	1	0	1	1		1
0	1	0	1	1	1		1
0	1	1	1	1	1	=	1
1	0	0	1	0	1		1
1	0	1	1	1	1		1
1	1	0	1	1	1		1
1	1	1	1	1	1		1

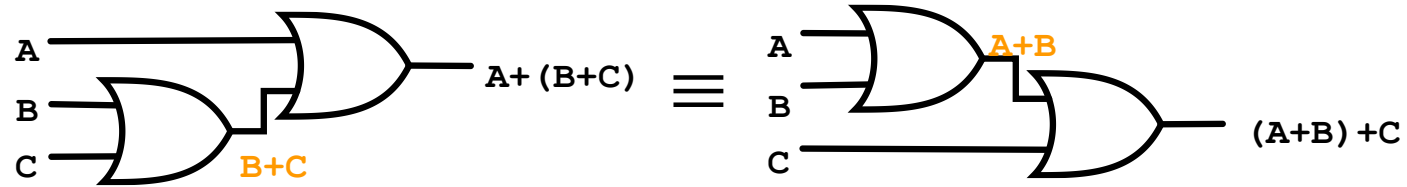
ASSOCIATIVE PROPERTY

- **Property2:** The associative property of multiplication states that, it makes no difference in what order the variables are grouped when AND'ing several variables. For three variables(A AND'ed B)AND'ed C is same as A AND'ed (B AND'ed C)
- i.e. $(A.B)C = A(B.C)$

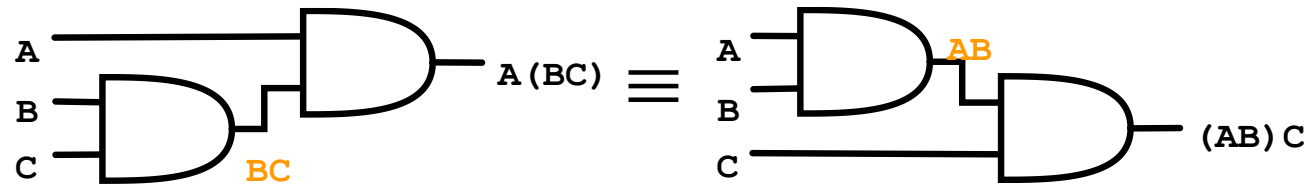
A	B	C	A.B	B.C	(A.B)C		A(B.C)
0	0	0	0	0	0		0
0	0	1	0	0	0		0
0	1	0	0	0	0		0
0	1	1	0	1	0	=	0
1	0	0	0	0	0		0
1	0	1	0	0	0		0
1	1	0	1	0	0		0
1	1	1	1	1	1		1

ASSOCIATIVE LAWS

- The *associative law of addition* for 3 variables is written as: $A+(B+C) = (A+B)+C$



- The *associative law of multiplication* for 3 variables is written as: $A(BC) = (AB)C$



DISTRIBUTIVE PROPERTY

Property 1: $A(B+C) = A.B + A.C$

1	2	3	4	5	6	7	8
A	B	C	B+C	A(B+C)	A.B	A.C	A.B+A.C
0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	1	0	0	0	0
0	1	1	1	0	0	0	0
1	0	0	0	0	0	0	0
1	0	1	1	1	0	1	1
1	1	0	1	1	1	0	1
1	1	1	1	1	1	1	1

Column number 5= Column number 8, hence the proof.

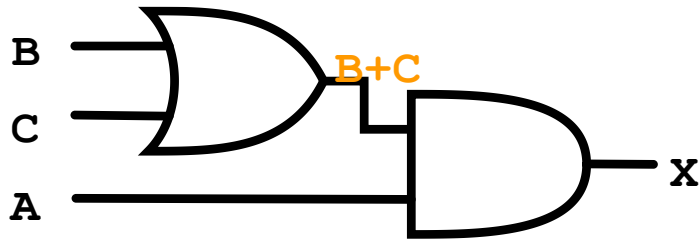
DISTRIBUTIVE PROPERTY

- Property 2: $A + A' B = A + B$

A	B	A'	A'B	A'B+A		A+B
0	0	1	0	0		0
0	1	1	1	1	=	1
1	0	0	0	1		1
1	1	0	0	1		1

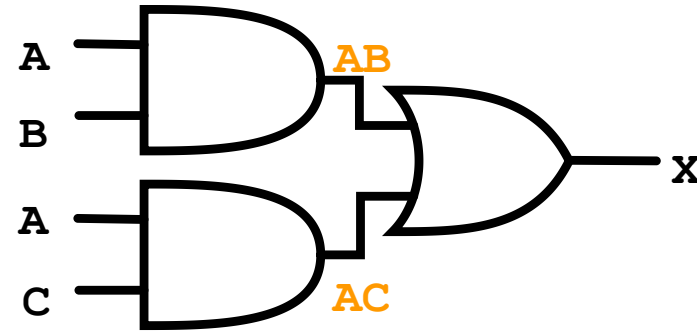
DISTRIBUTIVE LAWS

- The *distributive law* is written for 3 variables as follows: $A(B+C) = AB + AC$



$$X = A(B+C)$$

$\equiv$



$$X = AB + AC$$

RULES OF BOOLEAN ALGEBRA

$$1. A + 0 = A$$

$$2. A + 1 = 1$$

$$3. A \bullet 0 = 0$$

$$4. A \bullet 1 = A$$

$$5. A + A = A$$

$$6. A + \bar{A} = 1$$

$$7. A \bullet A = A$$

$$8. A \bullet \bar{A} = 0$$

$$9. \bar{\bar{A}} = A$$

$$10. A + AB = A$$

$$11. A + \bar{A}B = A + B$$

$$12. (A + B)(A + C) = A + BC$$

A, B, and C can represent a single variable or a combination of variables.

DUALITY

- The important property to Boolean algebra is called Duality principle. The Dual of any expression can be obtained easily by the following rules.
- 1. Change all 0's to 1's
- 2. Change all 1's to 0's
- 3. .'s (dot's) are replaced by +'s (plus)
- 4. +'s (plus) are replaced by .'s (dot's)

Examples:

$$X + 0 = X \equiv X \cdot 1 = X$$

$$X + Y = Y + X \equiv X \cdot Y = Y \cdot X$$

$$X + 1 = 1 \equiv X \cdot 0 = 0$$

Example Transformations

- Simple expression:

- Original: $A + 0 = A$

- Dual: $A \cdot 1 = A$

(OR becomes AND; 0 becomes 1)

- Algebraic identity duality:

- Original: $B + 1 = 1$

- Dual: $B \cdot 0 = 0$

Sanfoundry

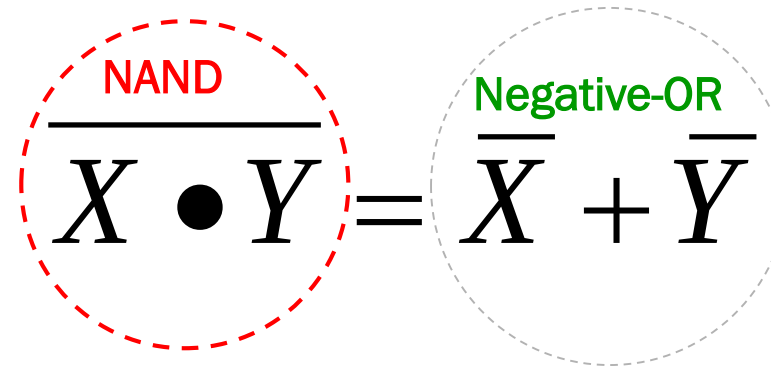
- Expression example:

- Given $A(B + C) = AB + AC$

- Dual: $A + (B \cdot C) = (A + B)(A + C)$

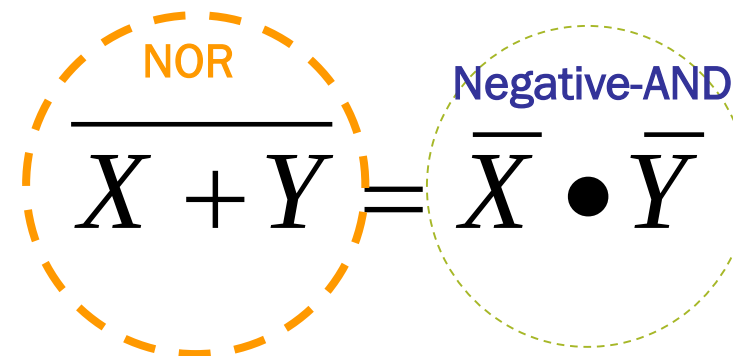
DEMORGAN'S THEOREMS

- The complement of two or more ANDed variables is equivalent to the OR of the complements of the individual variables.
- The complement of two or more ORed variables is equivalent to the AND of the complements of the individual variables.



The diagram illustrates the first De Morgan's theorem. On the left, the expression $\overline{X \bullet Y}$ is enclosed in a red dashed circle with the label "NAND" in red above it. This is followed by an equals sign, and then the expression $\overline{X} + \overline{Y}$ is enclosed in a grey dashed circle with the label "Negative-OR" in green above it.

$$\overline{X \bullet Y} = \overline{X} + \overline{Y}$$



The diagram illustrates the second De Morgan's theorem. On the left, the expression $\overline{X + Y}$ is enclosed in an orange dashed circle with the label "NOR" in orange above it. This is followed by an equals sign, and then the expression $\overline{X} \bullet \overline{Y}$ is enclosed in a green dashed circle with the label "Negative-AND" in blue above it.

$$\overline{X + Y} = \overline{X} \bullet \overline{Y}$$

DE MORGON'S FIRST THEOREM

- It states that “ the complements of product of two variables equal to sum of the complements of individual variable”. i.e. $(AB)' = A' + B'$

A	B	A'	B'	A.B	(AB)'		A'+B'
0	0	1	1	0	1		1
0	1	1	0	0	1	≡	1
1	0	0	1	0	1		1
1	1	0	0	1	0		0

DE MORGON'S SECOND THEOREM

- It states that complement of sum of two variables is equal to product of complement of two individual variables.

- $(A+B)' = A' \cdot B'$

A	B	A'	B'	A+B	(A+B)'		A' * B'
0	0	1	1	0	1		1
0	1	1	0	1	0	≡	0
1	0	0	1	1	0		0
1	1	0	0	1	0		0

DEMORGAN'S THEOREMS (EXERCISES)

- Apply DeMorgan's theorems to the expressions:

$$\overline{X \bullet Y \bullet Z}$$

$$\overline{X + Y + Z}$$

$$\overline{\bar{X} + \bar{Y} + \bar{Z}}$$

$$\overline{\bar{W} \bullet \bar{X} \bullet \bar{Y} \bullet \bar{Z}}$$

FUNCTION MINIMIZATION USING BOOLEAN ALGEBRA

- *Examples:*

$$(a) \ a + ab = a(1+b)=a$$

$$(b) \ a(a + b) = a.a +ab=a+ab=a(1+b)=a.$$

$$(c) \ a + a'b = (a + a')(a + b)=1(a + b) =a+b$$

$$(d) \ a(a' + b) = a.a' +ab=0+ab=ab$$

THE OTHER TYPE OF QUESTION

Show that;

$$1- ab + ab' = a$$

$$2- (a + b)(a + b') = a$$

$$1- ab + ab' = a(b+b') = a.1=a$$

$$\begin{aligned} 2- (a + b)(a + b') &= a.a + a.b' + a.b + b.b' \\ &= a + a.b' + a.b + 0 \\ &= a + a.(b' + b) + 0 \\ &= a + a.1 + 0 \\ &= a + a = a \end{aligned}$$

MORE EXAMPLES

- Show that;

$$(a) \quad ab + ab'c = ab + ac$$

$$(b) \quad (a + b)(a + b' + c) = a + bc$$

$$\begin{aligned}(a) \quad ab + ab'c &= a(b + b'c) \\ &= a((b+b').(b+c))=a(b+c)=ab+ac\end{aligned}$$

$$\begin{aligned}(b) \quad (a + b)(a + b' + c) \\ &= (a.a + a.b' + a.c + ab + b.b' + bc) \\ &= \dots\end{aligned}$$

SIMPLIFY THE BOOLEAN EXPRESSION

- 1. $X'Y'Z + X'YZ$
 $= ZX'[Y' + Y]$
 $= ZX'[1] = ZX'$
- 2. $f = B(A+C)+C$
 $= BA+BC+C$
 $= BA+C(1+B)$
 $= BA+C$

BOOLEAN ALGEBRA PROPERTIES

Let X: Boolean variable, 0,1:
constants

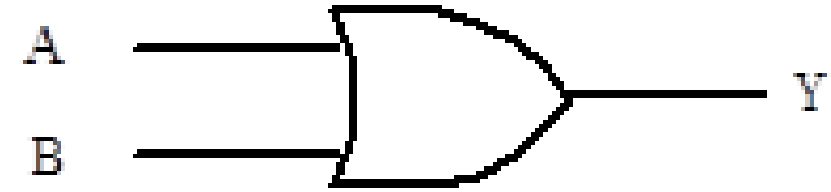
1. $X + 0 = X$ -- Zero Axiom
2. $X \cdot 1 = X$ -- Unit Axiom
3. $X + 1 = 1$ -- Unit Property
4. $X \cdot 0 = 0$ -- Zero Property
5. $X + X = X$ -- Idempotence
6. $X \cdot X = X$ -- Idempotence
7. $X + X' = 1$ -- Complement
8. $X \cdot X' = 0$ -- Complement
9. $(X')' = X$ -- Involution
5. $X + Y = Y + X$
6. $X \cdot Y = Y \cdot X$ -- Commutative
7. $X + (Y + Z) = (X + Y) + Z$
8. $X \cdot (Y \cdot Z) = (X \cdot Y) \cdot Z$ -- Associative
9. $X \cdot (Y + Z) = X \cdot Y + X \cdot Z$
10. $X + (Y \cdot Z) = (X + Y) \cdot (X + Z)$ -- Distributive
16. $(X + Y)' = X' \cdot Y'$
17. $(X \cdot Y)' = X' + Y'$ -- DeMorgan's

LOGIC GATES

- It is an electronic circuit, which makes logic decisions.
- A digital circuit is referred to as logic gate for simple reason i.e. it can be analysed based on Boolean algebra.
- To make logical decisions, three gates are used.
- They are OR, AND & NOT gate. These logic gates are building blocks, which are available in the form of IC.

OR GATE

- The OR gate performs logical additions commonly known as OR function.
- Two or more inputs and only one output.
- The operation of OR gate is such that a HIGH(1) on the output is produced when any of the input is HIGH.
- The output is LOW(0) only when all the inputs are LOW.

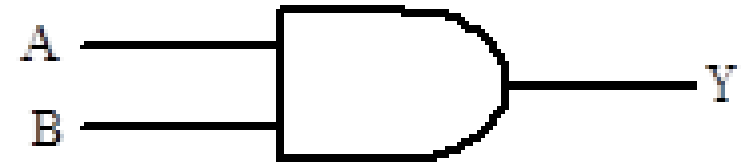


Input		Output
A	B	$Y = A + B$
0	0	0
0	1	1
1	0	1
1	1	1

Truth table for two input OR gate

AND GATE

- The AND gate performs logical multiplication.
- Two or more inputs and a single output.
- The output of an AND gate is HIGH only when all the inputs are HIGH.
- Even if any one of the input is LOW, the output will be LOW.

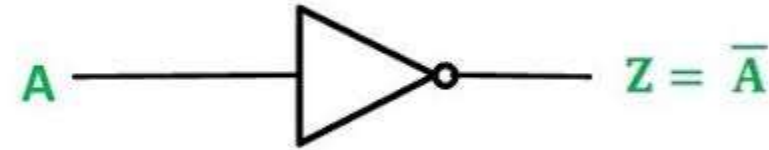


Input		Output
A	B	$Y=A.B$
0	0	0
0	1	0
1	0	0
1	1	1

Truth table for two input AND gate:

NOT GATE (INVERTER)

- The NOT gate performs the basic logical function called inversion or complementation.
- The purpose of this gate is to convert one logic level into the opposite logic level.
- It has one input and one output.

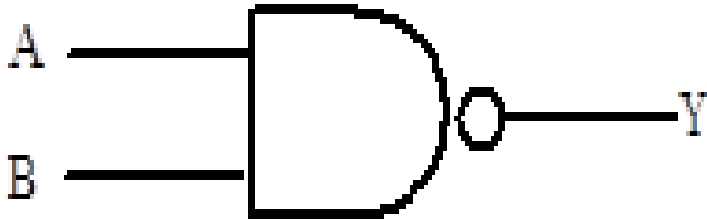


Input	output
A	$Z = A'$
0	1
1	0

Truth Table

NAND GATE

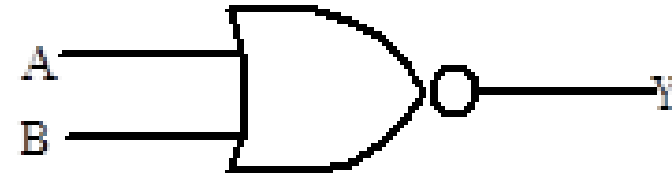
- The output of a NAND gate is LOW only when all inputs are HIGH and output of the NAND is HIGH if one or more inputs are LOW.



Input		Output
A	B	$Y = (AB)'$
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE

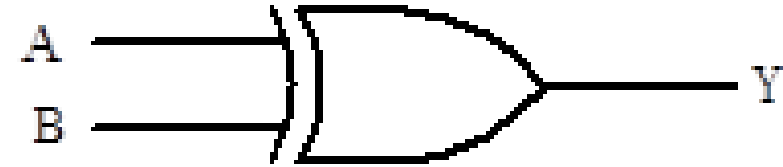
The output of the NOR gate is HIGH only when all the inputs are LOW.



Input		Output
A	B	$Y =$
0	0	1
0	1	0
1	0	0
1	1	0

XOR GATE OR EXCLUSIVE OR GATE

- In this gate output is HIGH only when any one of the input is HIGH.
- The circuit is also called as inequality comparator, because it produces output when two inputs are different.



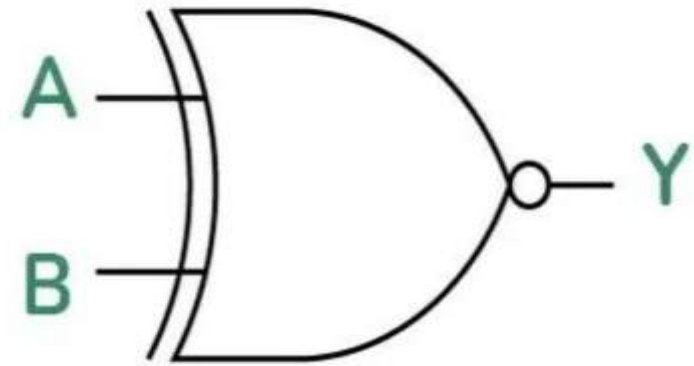
$$Y = A \oplus B = A\bar{B} + \bar{A}B$$

Input		Output
A	B	Y =
0	0	0
0	1	1
1	0	1
1	1	0

XNOR GATE LOGIC SYMBOL

Truth Table

Input A	Input B	Output
0	0	1
0	1	0
1	0	0
1	1	1

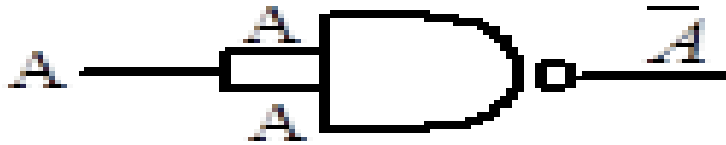


$$Y = A \odot B$$

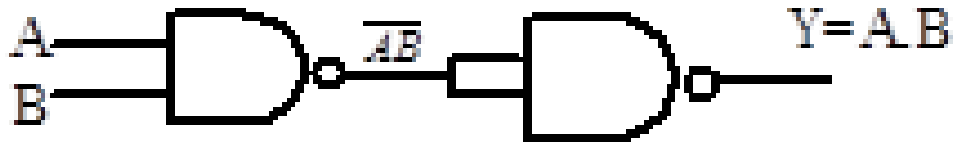
$$Y = (A \oplus B)'$$

UNIVERSAL LOGIC GATE

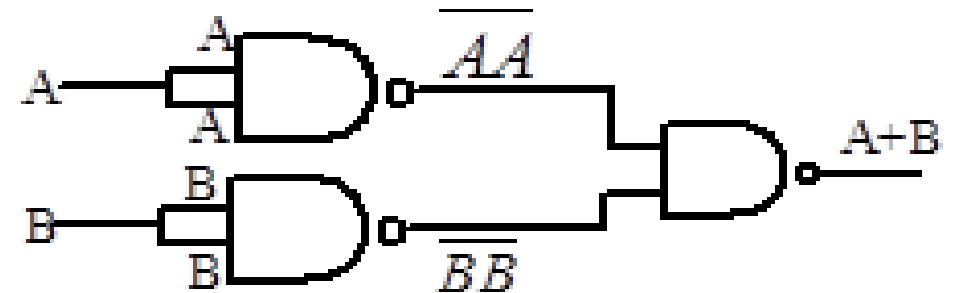
- NAND and NOR gates are called Universal gates or Universal building blocks, because both can be used to implement any gate like AND, OR and NOT gates or any combination of these basic gates.



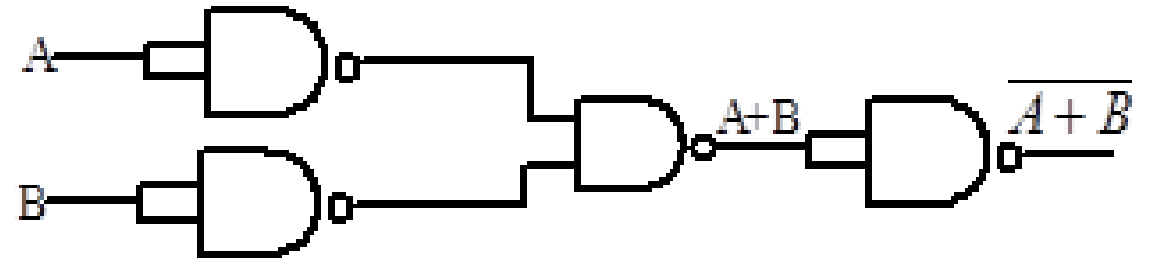
NOT operation:



AND operation:



OR operation:



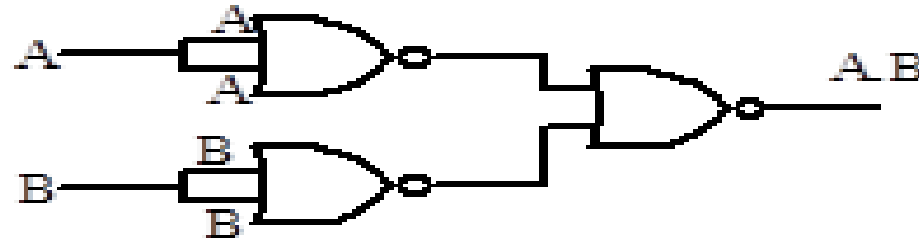
NOR operation:

NOR GATE AS UNIVERSAL GATE:

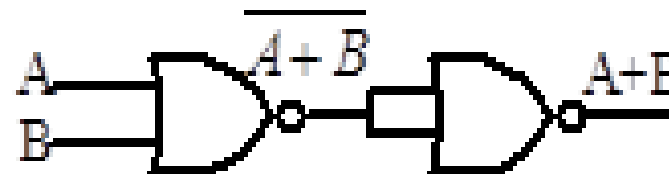
NOT operation:



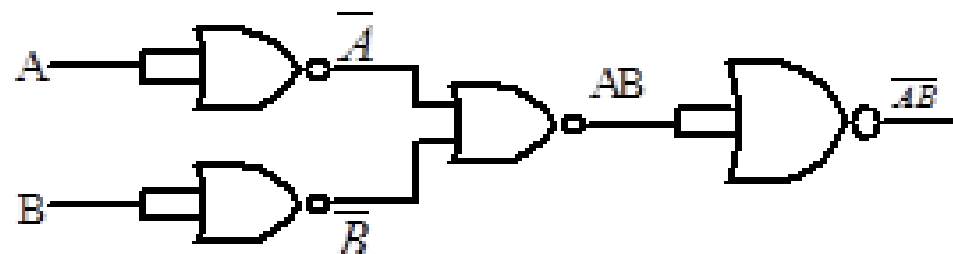
AND operation:



OR operation:



NAND operation:



Standard Representation

- Any logical expression can be expressed in the following two forms:
 - ✓ Sum of Product (SOP) Form
 - ✓ Product of Sum (POS) Form

SOP Form

For Example, logical expression given is;

$$Y = A.B + B.C + A.C$$

Sum

Product

POS Form

For Example, logical expression given is;

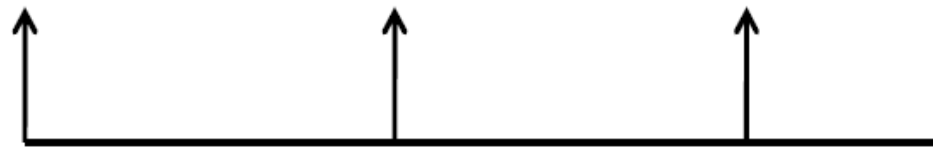
$$Y = (A + B).(B + C).(A + C)$$

Product

Sum

Standard SOP

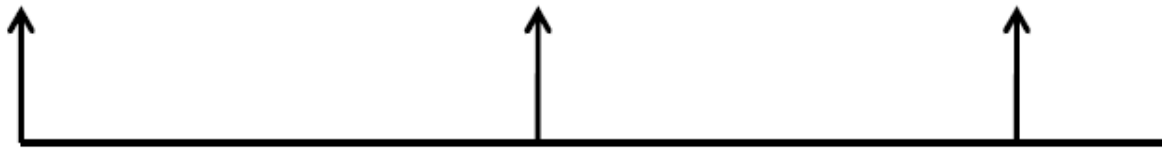
$$Y = ABC + \overline{A}\overline{B}\overline{C} + \overline{A}BC$$



Each product term
consists all the
literals

Standard POS

$$Y = (A + B + C).(A + \overline{B} + \overline{C}).(\overline{A} + B + C)$$



Each sum term
consists all the
literals

Sr. No.	Expression	Type
1	$Y = AB + AB\overline{C} + \overline{A}BC$	Non Standard SOP
2	$Y = AB + A\overline{B} + \overline{A}\overline{B}$	Standard SOP
3	$Y = (\overline{A} + B).(A + \overline{B}).(\overline{A} + \overline{B})$	Standard POS
4	$Y = (\overline{A} + B).(A + \overline{B} + C)$	Non Standard POS

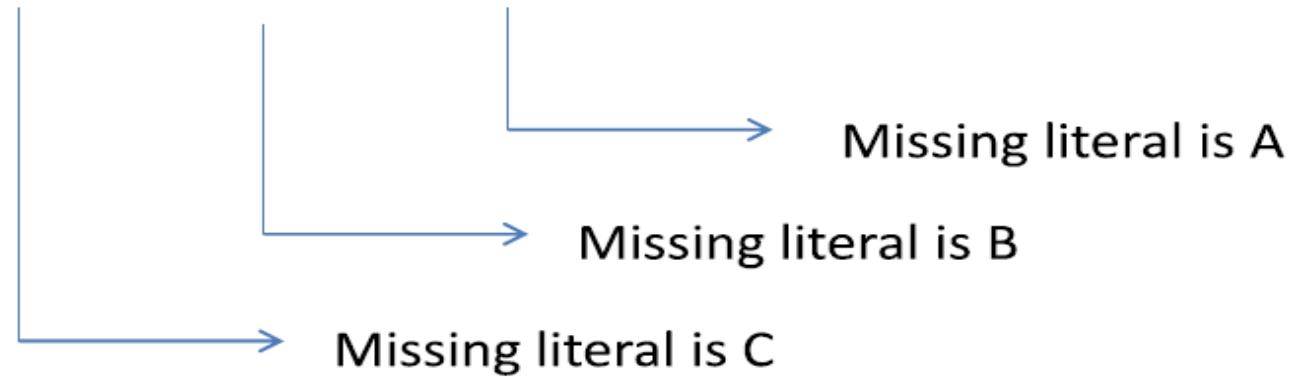
Conversion of SOP form to Standard SOP

Procedure:

1. Write down all the terms.
2. If one or more variables are missing in any product term, expand the term by multiplying it with the sum of each one of the missing variable and its complement .
3. Drop out the redundant terms

Convert given expression into its standard SOP form $Y = AB + A\bar{C} + BC$

$$Y = AB + A\bar{C} + BC$$



$$Y = AB.(C + \bar{C}) + A\bar{C}.(B + \bar{B}) + BC.(A + \bar{A})$$

Term formed by ORing of missing literal & its complement

Example 1

Continue....

$$Y = AB.(C + \overline{C}) + A\overline{C}.(B + \overline{B}) + BC.(A + \overline{A})$$

$$Y = ABC + AB\overline{C} + A\overline{B}\overline{C} + \overline{A}BC + A\overline{B}C + \overline{A}B\overline{C}$$

$$Y = \underline{ABC} + \underline{AB\overline{C}} + \underline{A\overline{B}\overline{C}} + \underline{\overline{A}BC} + \overline{A}B\overline{C} + \overline{A}B\overline{C}$$

$$Y = \underline{ABC + AB\overline{C} + A\overline{B}\overline{C} + \overline{A}BC}$$

Standard SOP form

Each product term consists all the literals

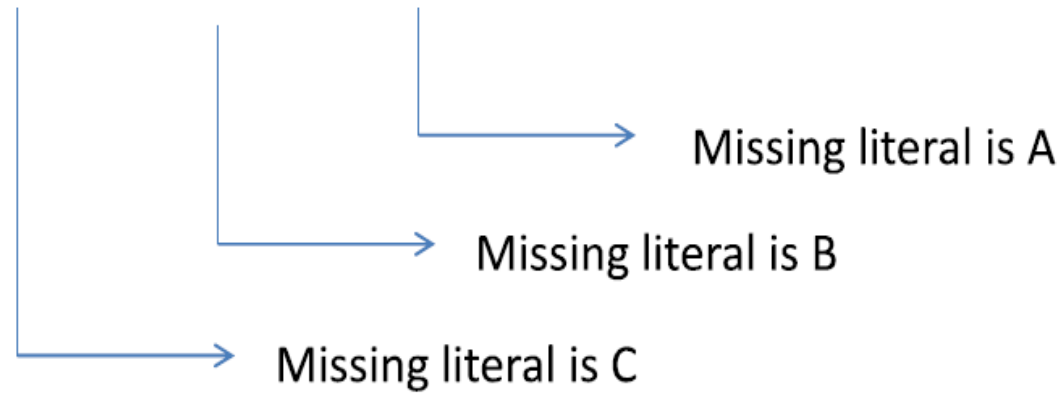
Conversion of POS form to Standard POS

Procedure:

1. Write down all the terms.
2. If one or more variables are missing in any sum term, expand the term by adding the products of each one of the missing variable and its complement .
3. Drop out the redundant terms

Convert given expression into its standard SOP form $Y = (A + B).(A + C) + (B + \bar{C})$

$$Y = (A + B).(A + C) + (B + \bar{C})$$



$$Y = (A + B + \underline{C\bar{C}}).(A + C + \underline{B\bar{B}}).(B + \bar{C} + \underline{A\bar{A}})$$

Term formed by ANDing of missing literal & its complement

$$Y = (A + B + C\overline{C}).(A + C + B\overline{B}).(B + \overline{C} + A\overline{A})$$

$$Y = (\underline{A+B+C})(\underline{A+B+\overline{C}}).(\underline{A+B+C})(A+\overline{B}+C).(\underline{A+B+\overline{C}})(\overline{A}+B+\overline{C})$$

$$Y = (A+B+C)(A+B+\overline{C})(A+\overline{B}+C)(\overline{A}+B+\overline{C})$$

$$Y = (A+B+C)(A+B+\overline{C})(A+\overline{B}+C)(\overline{A}+B+\overline{C})$$

Standard POS form

Each sum term consists all the literals

Concept of Minterm and Maxterm

- Minterm: Each individual term in the standard SOP form is called as “Minterm”.
- Maxterm: Each individual term in the standard POS form is called as “Maxterm”.

B.TECH FIRST YEAR

ACADEMIC YEAR: 2025-2026



COURSE NAME: BASIC ELECTRONICS

COURSE CODE : ECE1001

LECTURE SERIES NO : 01(ONE)

CREDITS : 3

MODE OF DELIVERY : ONLINE (POWER POINT PRESENTATION)

FACULTY :

EMAIL-ID :

PROPOSED DATE OF DELIVERY:





SESSION OUTCOME

"A SYSTEMATIC WAY TO MINIMIZE
THE GIVEN LOGIC "



ASSIGNMENT

QUIZ

MID TERM EXAMINATION –II

END TERM EXAMINATION

ASSESSMENT CRITERIA'S



PROGRAM OUTCOMES MAPPING WITH CO2

[PO1]

DEMONSTRATE DIFFERENT NUMBER SYSTEMS, BOOLEAN EXPRESSIONS AND DIFFERENT ELEMENTS OF COMMUNICATION SYSTEMS AND TO PROMOTE DIFFERENT SKILLS TOWARDS ELECTRONICS INDUSTRIES.

KARNAUGH MAP

The Karnaugh map, also known as the K-map, is a method to simplify Boolean algebra expressions.

The Karnaugh map reduces the need for extensive calculations by taking advantage of humans' pattern-recognition capability.

Karnaugh Map: A graphical technique for simplifying a Boolean expression into either form:

- minimal sum of products
- minimal product of sums

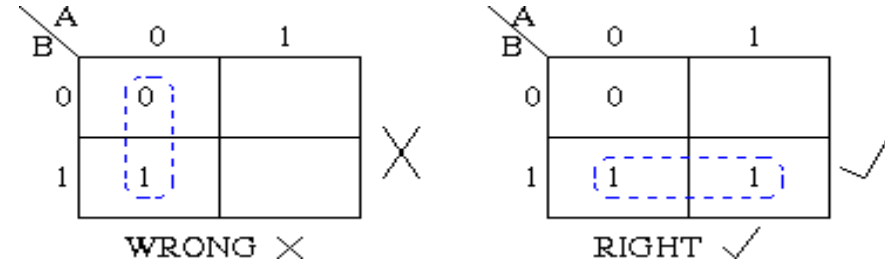
Goal of the simplification.

- There are a minimal number of product/sum terms
- Each term has a minimal number of literals

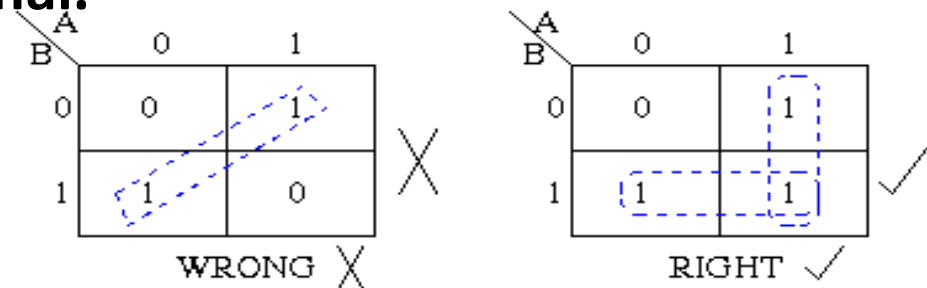
These terms can be used to write a minimal Boolean expression representing the required logic.

THE KARNAUGH MAP USES THE FOLLOWING RULES FOR THE SIMPLIFICATION OF EXPRESSIONS BY GROUPING TOGETHER ADJACENT CELLS CONTAINING *ONES*.

1.Groups may not include any cell containing a zero.



2.Groups may be horizontal or vertical, but not diagonal.



3. GROUPS MUST CONTAIN 1, 2, 4, 8 CELLS OR IN GENERAL 2^N CELLS. THAT IS IF $N = 1$, A GROUP WILL CONTAIN TWO 1'S SINCE $2^1 = 2$. IF $N = 2$, A GROUP WILL CONTAIN FOUR 1'S SINCE $2^2 = 4$.

		A	
		0	1
B	0	1	1
	1	0	0

Group of 2 ✓

RIGHT ✓

		AB			
		00	01	11	10
C	0	0	1	1	1
	1	0	0	0	0

Group of 3 ✗

WRONG ✗

		A	
		0	1
B	0	1	1
	1	1	1

Group of 4 ✓

RIGHT ✓

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	0	0	1

Group of 5 ✗

WRONG ✗

4. EACH GROUP SHOULD BE AS LARGE AS POSSIBLE.

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

RIGHT ✓

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

WRONG ✗

(Note that no Boolean laws broken,
but not sufficiently minimal)

5. Each cell containing a one must be in at least one group.

		AB			
		00	01	11	10
C	0	0	0	1	1
	1	0	0	0	1

Group I
Group II

1 present in at least one group.

6.GROUPS MAY OVERLAP.

AB C		00 01 11 10			
		00	01	11	10
0	1	1	1	1	
1	0	0	1	1	

Groups overlapping. ✓

RIGHT ✓

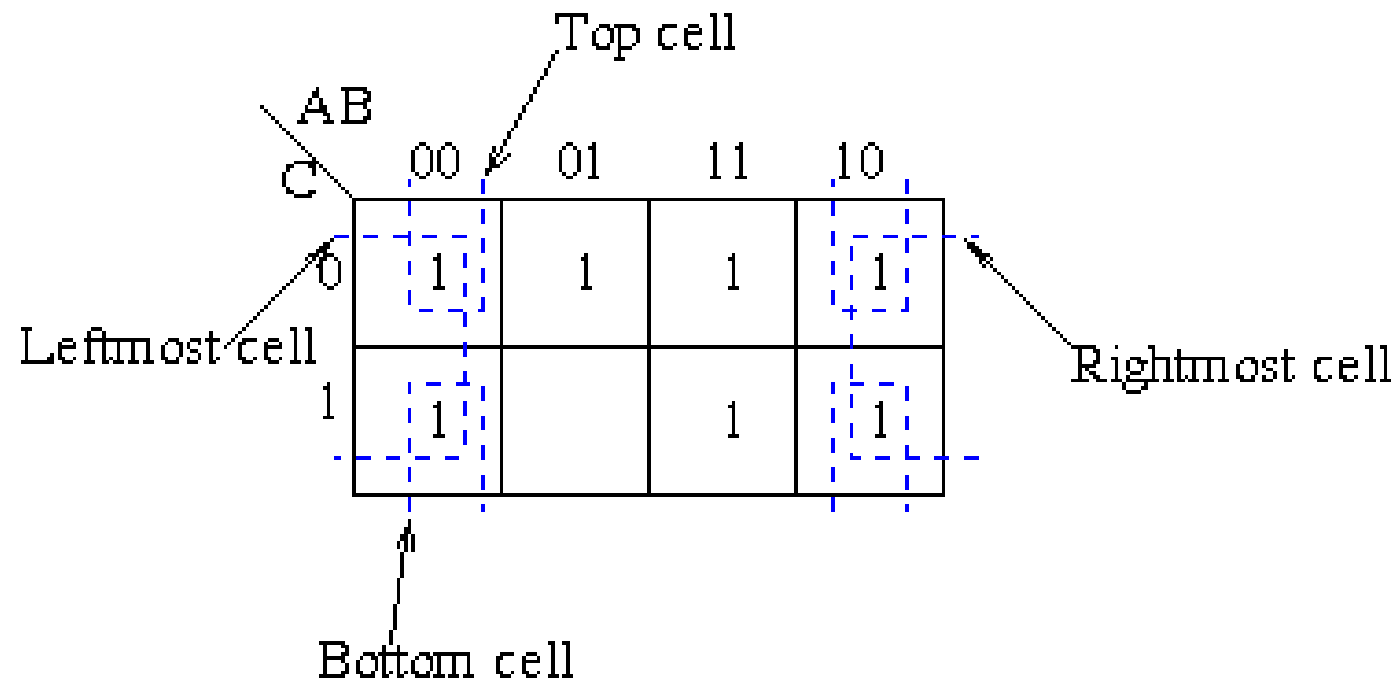
AB \ C		00 01 11 10			
		00	01	11	10
0	1	1	1	1	
1	0	0	1	1	

Groups not overlapping.

WRONG ✗

7.Groups may wrap around the table.

The leftmost cell in a row may be grouped with the rightmost cell and the top cell in a column may be grouped with the bottom cell.



8. THERE SHOULD BE AS FEW GROUPS AS POSSIBLE, AS LONG AS THIS DOES NOT CONTRADICT ANY OF THE PREVIOUS RULES.

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

RIGHT ✓

		AB			
		00	01	11	10
C	0	1	1	1	1
	1	0	0	1	1

WRONG ✗



SUMMARY:

- No zeros allowed.
- No diagonals.
- Only power of 2 number of cells in each group.
- Groups should be as large as possible.
- Every one must be in at least one group.
- Overlapping allowed.
- Wrap around allowed.
- Fewest number of groups possible.

KARNAUGH MAPS

- Karnaugh maps, or K-maps, are often used to simplify logic problems with 2, 3 or 4 variables.

Cell = 2^n ,where n is a number of variables

For the case of 2 variables, we form a map consisting of $2^2=4$ cells as shown in Figure

		A	
		0	1
B	0	$A + B$	$\bar{A} + B$
	1	$A + \bar{B}$	$\bar{A} + \bar{B}$

Maxterm

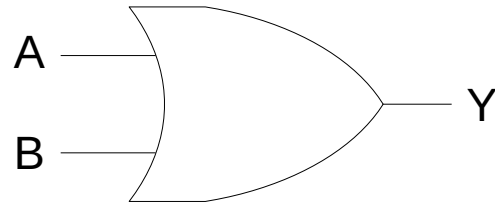
		A	
		0	1
B	0	00 0	10 2
	1	01 1	11 3

		A	
		0	1
B	0	$\bar{A}\bar{B}$	$A\bar{B}$
	1	$\bar{A}B$	AB

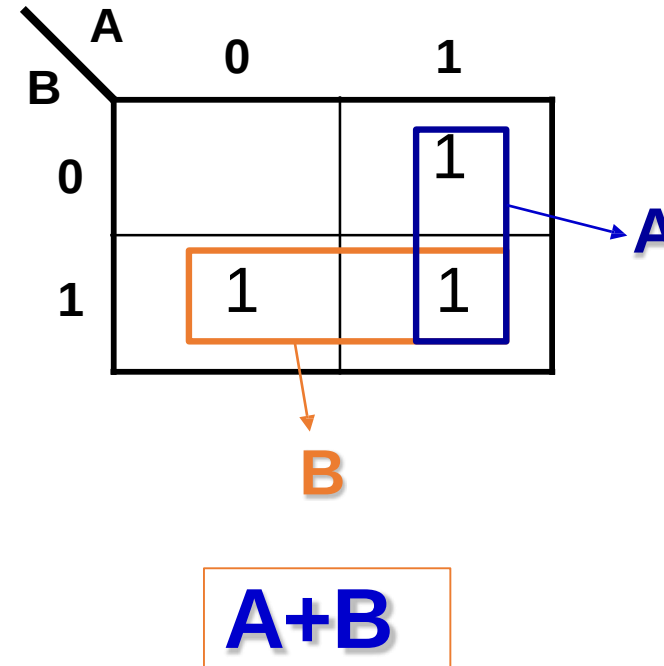
Minterm

EXAMPLE

2-variable Karnaugh maps are trivial but can be used to introduce the methods you need to learn. The map for a 2-input OR gate looks like this:



A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



EXAMPLE 1: Consider the following map. The function plotted is: $Z = f(A,B) = AB' + AB$

B \ A	0	1
0		1
1		1

Note that values of the input variables form the rows and columns.

That is the logic values of the variables A and B (with one denoting true form and zero denoting false form) form the head of the rows and columns respectively.

The map displayed is a one dimensional type which can be used to simplify an expression in two variables.

There is a two-dimensional map that can be used for up to four variables, and a three-dimensional map for up to six variables.



REFERRING TO THE MAP ABOVE, THE TWO ADJACENT 1'S ARE GROUPED TOGETHER.

Through inspection it can be seen that variable B has its true and false form within the group.

This eliminates variable B leaving only variable A which only has its true form.

The minimized answer therefore is $Z = A$.

KARNAUGH MAPS

- 3 variables Karnaugh map

$$\text{Cell} = 2^3 = 8$$

		AB				
		C	00	01	11	10
0		0	⁰ $\bar{A}\bar{B}\bar{C}$	² $\bar{A}B\bar{C}$	⁶ $AB\bar{C}$	⁴ $A\bar{B}\bar{C}$
		1 <td>¹ $\bar{A}\bar{B}C$</td> <td>³ $\bar{A}BC$</td> <td>⁷ ABC</td> <td>⁵ $A\bar{B}C$</td>	¹ $\bar{A}\bar{B}C$	³ $\bar{A}BC$	⁷ ABC	⁵ $A\bar{B}C$

THREE-VARIABLE K-MAP

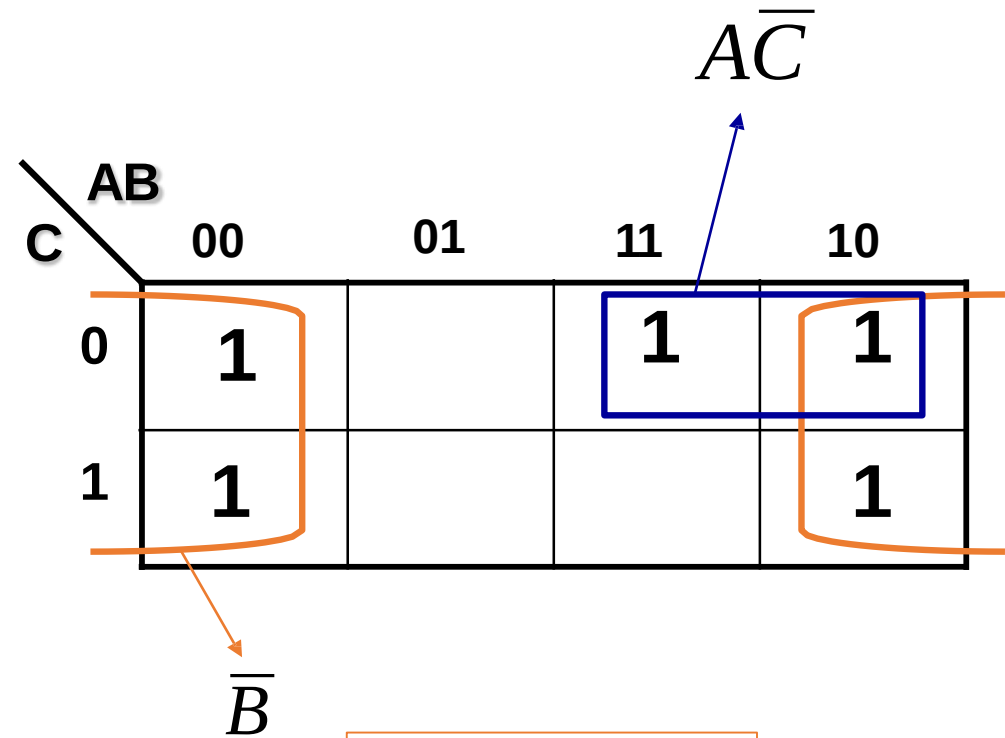
ab \ c		ab			
		00	01	11	10
c	0	m_0	m_2	m_6	m_4
	1	m_1	m_3	m_7	m_5

GUIDELINES FOR SIMPLIFYING FUNCTIONS

- Group as many squares as possible. This eliminates the most variables.
- Make as few groups as possible. Each group represents a separate product term.
- You must **cover** each minterm at least once. However, it may be covered more than once.

EXAMPLE

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0



$$\overline{B} + A\overline{C}$$

Three-Variable K-Map Example

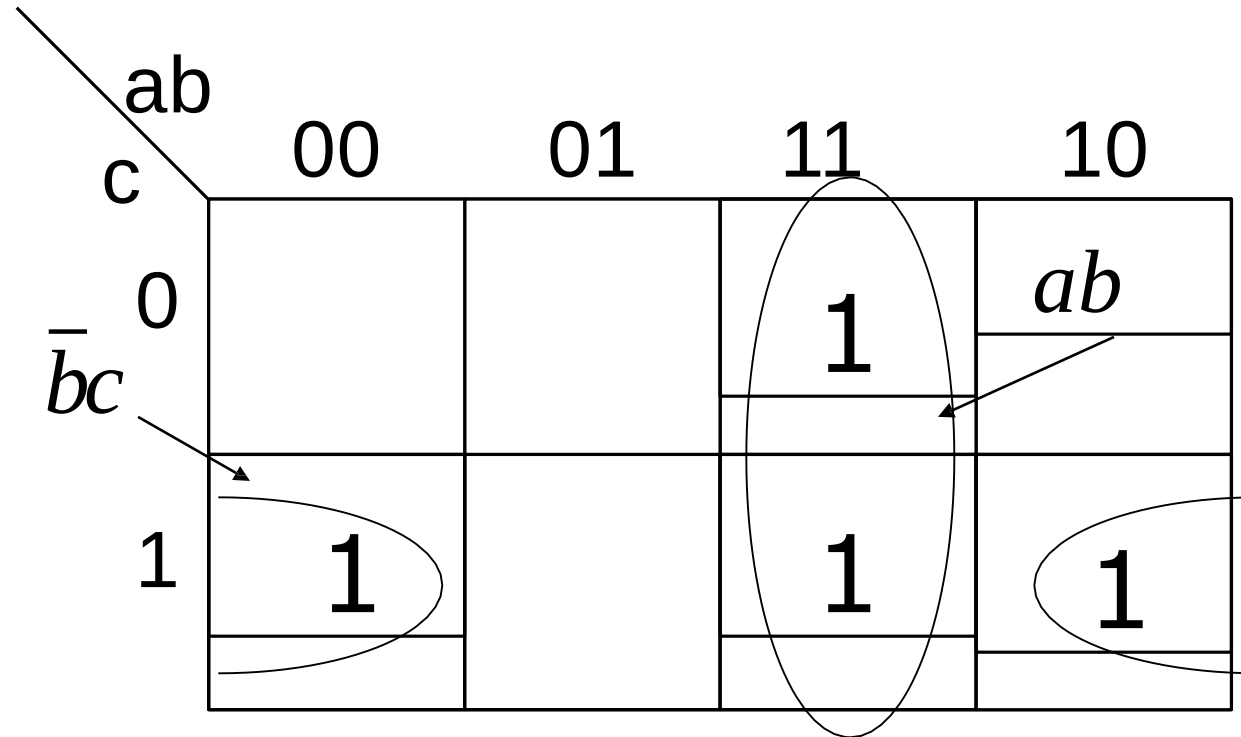
Plot 1's (minterms) of switching function

ab \ c		ab			
		00	01	11	10
c	0			1	
	1	1	1		1

$$F(a,b,c) = \sum m(1,3,5,6)$$

Three-Variable K-Map Example

Plot 1's (minterms) of switching function



$$F(a, b, c) = ab + \bar{b}c$$

THREE-VARIABLE K-MAPS

$$f = \sum (0,4) = \overline{B}\overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	0
	1	1	0	0	0

$$f = \sum (4,5) = A\overline{B}$$

		BC			
		00	01	11	10
A	0	0	0	0	0
	1	1	1	0	0

$$f = \sum (0,1,4,5) = \overline{B}$$

		BC			
		00	01	11	10
A	0	1	1	0	0
	1	1	1	0	0

$$f = \sum (0,1,2,3) = \overline{A}$$

		BC			
		00	01	11	10
A	0	1	1	1	1
	1	0	0	0	0

$$f = \sum (0,4) = \overline{A}\overline{C}$$

		BC			
		00	01	11	10
A	0	0	1	1	0
	1	0	0	0	0

$$f = \sum (4,6) = A\overline{C}$$

		BC			
		00	01	11	10
A	0	0	0	0	0
	1	1	0	0	1

$$f = \sum (0,2) = \overline{A}\overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	1
	1	0	0	0	0

$$f = \sum (0,2,4,6) = \overline{C}$$

		BC			
		00	01	11	10
A	0	1	0	0	1
	1	1	0	0	1

THREE-VARIABLE K-MAP EXAMPLES

A \ BC	BC			
	00	01	11	10
0		1		
1	1		1	1

A \ BC	BC			
	00	01	11	10
0	1		1	1
1	1			1

A \ BC	BC			
	00	01	11	10
0			1	1
1	1	1		

A \ BC	BC			
	00	01	11	10
0			1	
1	1		1	1

A \ BC	BC			
	00	01	11	10
0		1	1	1
1		1	1	

A \ BC	BC			
	00	01	11	10
0	1		1	
1		1	1	1

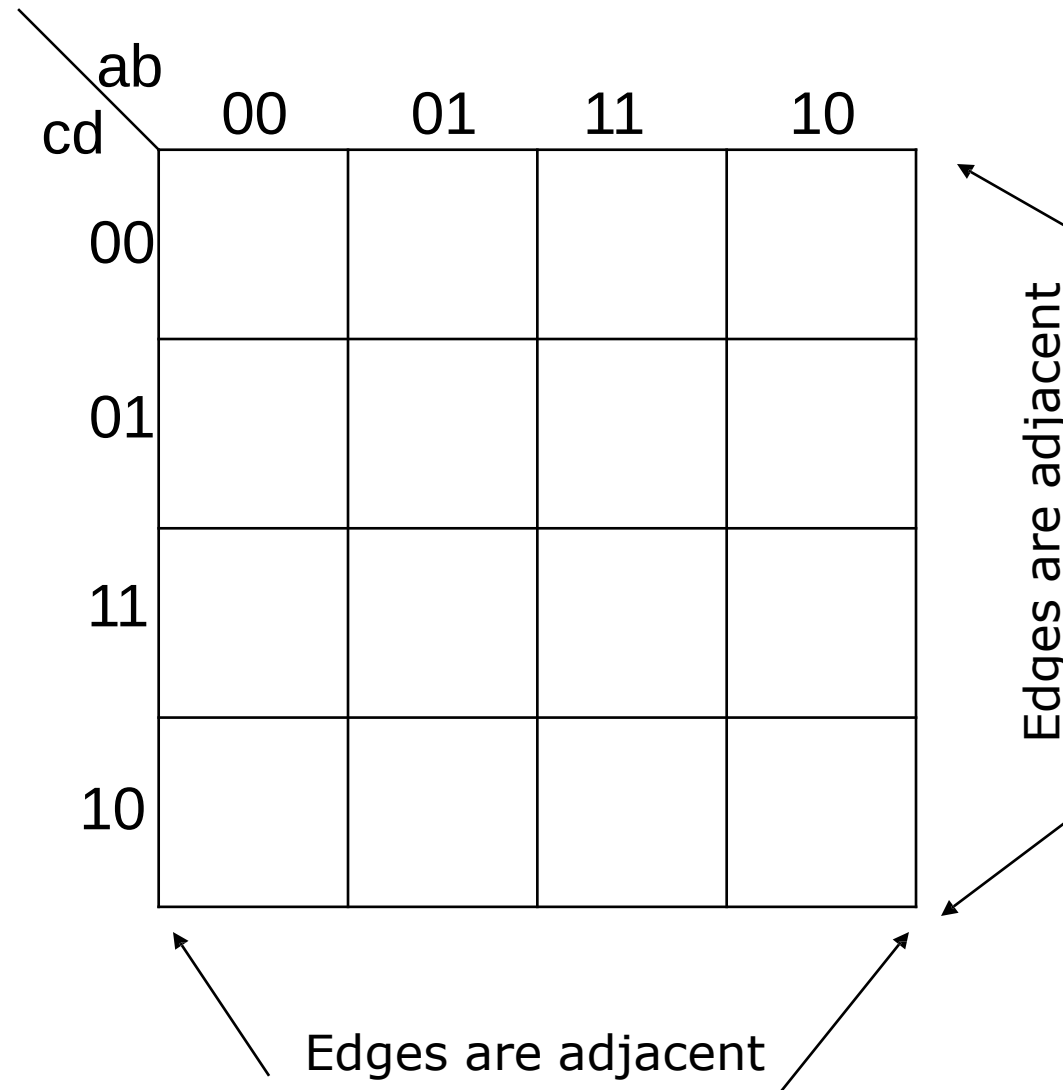


FOUR VARIABLE EXAMPLES

FOUR-VARIABLE K-MAP

		ab			
		00	01	11	10
cd	00	m_0	m_4	m_{12}	m_8
	01	m_1	m_5	m_{13}	m_9
	11	m_3	m_7	m_{15}	m_{11}
	10	m_2	m_6	m_{14}	m_{10}

FOUR-VARIABLE K-MAP



Example

- Use a K-Map to simplify the following Boolean expression

$$F(a, b, c, d) = \sum m(0, 2, 3, 6, 8, 12, 13, 15)$$

Four-variable K-Map

A 4x4 Karnaugh map for four variables a, b, c, and d. The columns are labeled 'ab' with values 00, 01, 11, 10. The rows are labeled 'cd' with values 00, 01, 11, 10. The map contains 1s in the following cells: (00,00), (11,00), (10,00), (11,01), (11,11), (00,10), (01,10), (11,10), and (10,10). These 1s are grouped into eight pairs, each circled with an ellipse: (00,00) and (11,00) are grouped; (11,00) and (10,00) are grouped; (11,01) and (11,11) are grouped; (00,10) and (01,10) are grouped; (11,11) and (11,10) are grouped; (00,10) and (11,10) are grouped; (11,10) and (10,10) are grouped; and (00,10) and (10,10) are grouped.

cd \ ab	00	01	11	10
00	1		1	1
01			1	
11	1		1	
10	1	1		

$$F = \sum m(0,2,3,6,8,12,13,15)$$

Four-variable K-Map

		ab			
		00	01	11	10
cd	00	1		1	1
	01			1	
	11	1		1	
	10	1	1		

$$F = \overline{a}\overline{b}\overline{c}d + \overline{a}b\overline{c}c + \overline{a}c\overline{d} + ab\overline{c}d + a\overline{c}\overline{d}$$

FOUR-VARIABLE K- MAPS

		CD			
		00	01	11	10
AB	00	1	0	0	0
	01	0	0	0	0
	11	0	0	0	0
	10	1	0	0	0

$$f = \sum(0,8) = \bar{B} \cdot \bar{C} \cdot \bar{D}$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	0	1	0	0
	11	0	1	0	0
	10	0	0	0	0

$$f = \sum(5,13) = B \cdot \bar{C} \cdot D$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	0	0	0	0
	11	0	1	1	0
	10	0	0	0	0

$$f = \sum(13,15) = A \cdot B \cdot D$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	1	0	0	1
	11	0	0	0	0
	10	0	0	0	0

$$f = \sum(4,6) = \bar{A} \cdot B \cdot \bar{D}$$

		CD			
		00	01	11	10
AB	00	0	0	1	1
	01	0	0	1	1
	11	0	0	0	0
	10	0	0	0	0

$$f = \sum(2,3,6,7) = \bar{A} \cdot C$$

		CD			
		00	01	11	10
AB	00	0	0	0	0
	01	1	0	0	1
	11	1	0	0	1
	10	0	0	0	0

$$f = \sum(4,6,12,14) = B \cdot \bar{D}$$

		CD			
		00	01	11	10
AB	00	0	0	1	1
	01	0	0	0	0
	11	0	0	0	0
	10	0	0	1	1

$$f = \sum(2,3,10,11) = \bar{B} \cdot C$$

		CD			
		00	01	11	10
AB	00	1	0	0	1
	01	0	0	0	0
	11	0	0	0	0
	10	1	0	0	1

$$f = \sum(0,2,8,10) = \bar{B} \cdot \bar{D}$$

FOUR-VARIABLE K- MAPS

AB \ CD	CD			
	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	0	0	0	0
10	0	0	0	0

$$f = \sum(4,5,6,7) = \bar{A} \bullet B$$

AB \ CD	CD			
	00	01	11	10
00	0	0	1	0
01	0	0	1	0
11	0	0	1	0
10	0	0	1	0

$$f = \sum(3,7,11,15) = C \bullet D$$

AB \ CD	CD			
	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

$$f = \sum(0,3,5,6,9,10,12,15)$$

$$f = A \otimes B \otimes C \otimes D$$

AB \ CD	CD			
	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

$$f = \sum(1,2,4,7,8,11,13,14)$$

$$f = A \oplus B \oplus C \oplus D$$

AB \ CD	CD			
	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

$$f = \sum(1,3,5,7,9,11,13,15)$$

$$f = D$$

AB \ CD	CD			
	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	1	0	0	1
10	1	0	0	1

$$f = \sum(0,2,4,6,8,10,12,14)$$

$$f = \bar{D}$$

AB \ CD	CD			
	00	01	11	10
00	0	0	0	0
01	1	1	1	1
11	1	1	1	1
10	0	0	0	0

$$f = \sum(4,5,6,7,12,13,14,15)$$

$$f = B$$

AB \ CD	CD			
	00	01	11	10
00	1	1	1	1
01	0	0	0	0
11	0	0	0	0
10	1	1	1	1

$$f = \sum(0,1,2,3,8,9,10,11)$$

$$f = \bar{B}$$

FOUR-VARIABLE K-MAPS

EXAMPLES

		CD			
		00	01	11	10
AB	00	1	1		1
	01	1	1		1
	11	1	1		1
	10	1	1		

		CD			
		00	01	11	10
AB	00	1	1		1
	01				1
	11				
	10	1	1		1

		CD			
		00	01	11	10
AB	00				
	01	1	1	1	
	11	1	1		1
	10	1			

		CD			
		00	01	11	10
AB	00		1	1	
	01	1	1	1	1
	11	1		1	1
	10			1	

		CD			
		00	01	11	10
AB	00				
	01				
	11				
	10				

		CD			
		00	01	11	10
AB	00				
	01				
	11				
	10				

Example

- Use a K-Map to simplify the following Boolean expression

$$F(a, b, c) = \sum m(2, 3, 6, 7)$$

Three-Variable K-Map Example

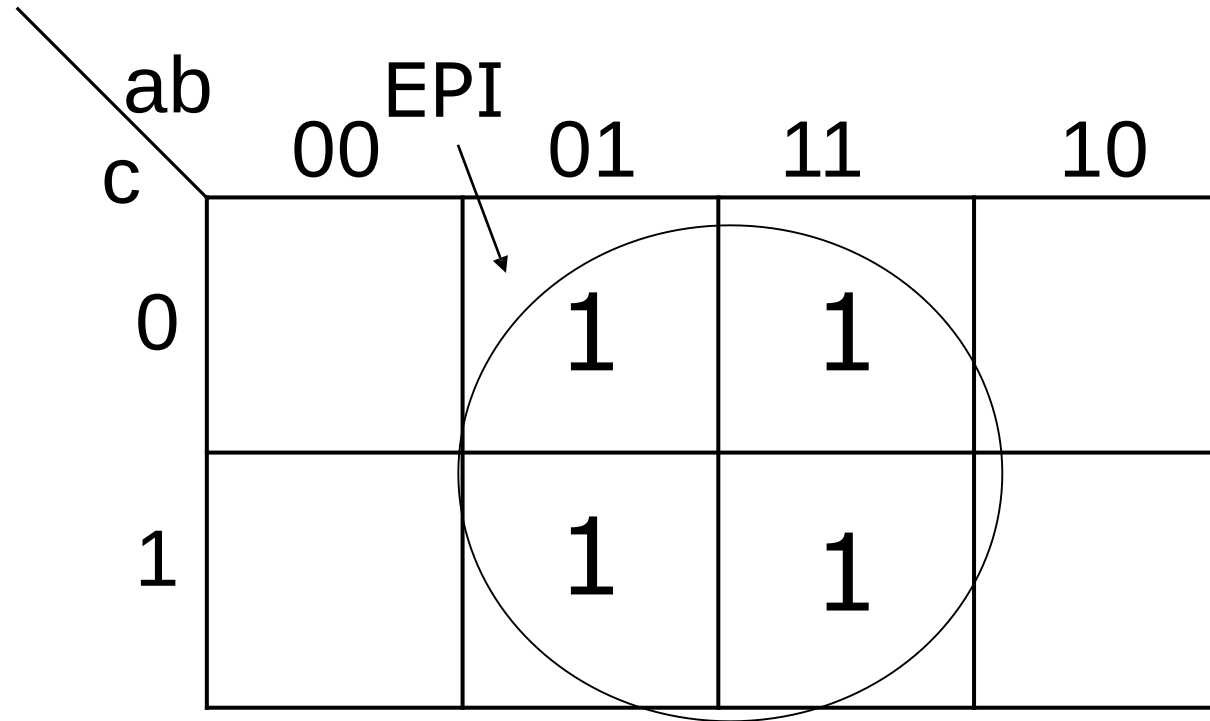
Step 1: Plot the K-map

ab \ c		ab			
		00	01	11	10
c	0		1	1	
	1		1	1	

$$F(a,b,c) = \sum m(2,4,5,7)$$

Three-Variable K-Map Example

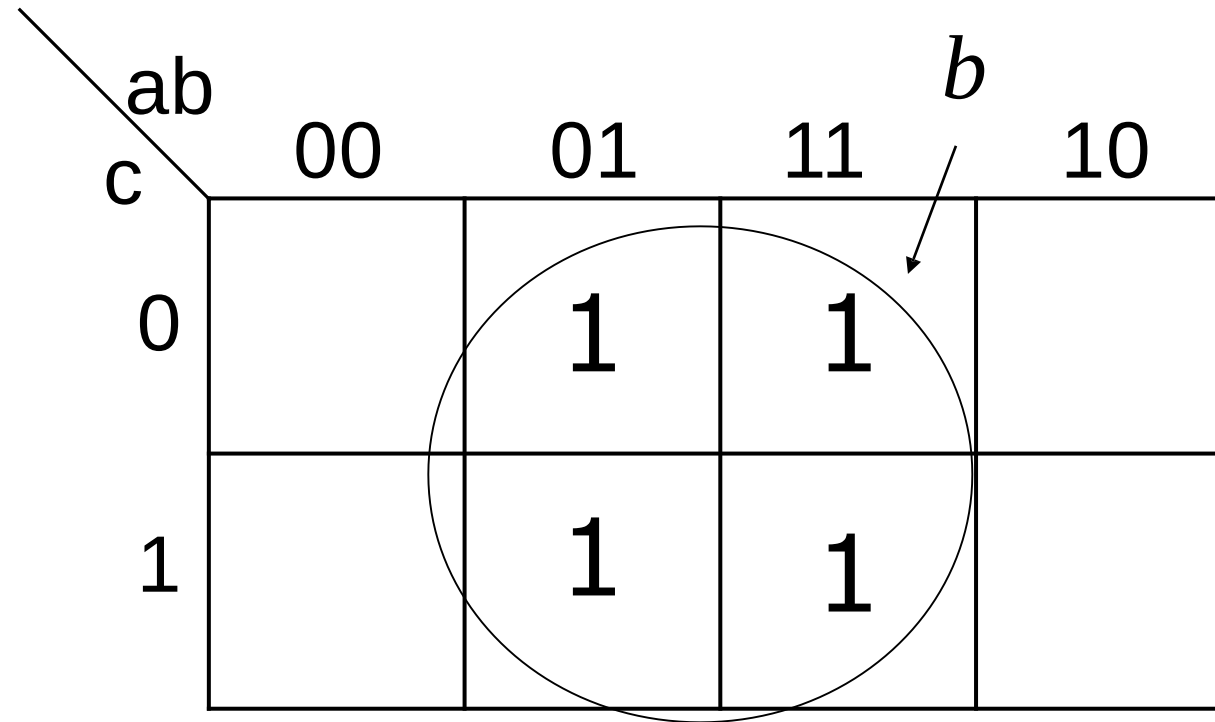
Step 3: Identify Essential Prime Implicants



$$F(a,b,c) = \sum m(2,3,6,7)$$

Three-Variable K-Map Example

Step 5: Read the map.



$$F(a,b,c) = \sum m(2,3,6,7)$$

Solution

$$F(a, b, c) = b$$



SPECIAL CASES

THREE-VARIABLE K-MAP

EXAMPLE

ab \ c		ab			
		00	01	11	10
c	0	1	1	1	1
	1	1	1	1	1

$$F(a,b,c)=1$$

THREE-VARIABLE K-MAP EXAMPLE

		ab			
		00	01	11	10
c	0	0	0	0	0
	1	0	0	0	0

$$F(a, b, c) = 0$$

THREE-VARIABLE K-MAP EXAMPLE

		ab			
		00	01	11	10
c	0		1		1
	1	1		1	

$$F(a,b,c) = a \oplus b \oplus c$$

Example

- Use a K-Map to simplify the following Boolean expression

$$F(a, b, c) = \sum m(1, 2, 3, 5, 6)$$

Three-Variable K-Map Example

Step 1: Plot the K-map

ab \ c		ab			
		00	01	11	10
c	0		1	1	
	1	1	1		1

$$F(a,b,c) = \sum m(1,2,3,5,6)$$



B.TECH FIRST YEAR

ACADEMIC YEAR: 2025-2026

Course name: Digital system

COURSE CODE : ECE1001

LECTURE SERIES NO :

CREDITS : 3

**MODE OF DELIVERY :
PRESENTATION)**

**FACULTY : DR.SUMIT
GUPTA**



Session outcome

"DISCUSS AND
UNDERSTAND
VARIOUS
COMBINATION
AL LOGIC
CIRCUITS"

ASSIGNMENT

QUIZ

MID TERM
EXAMINATION
-II

END TERM
EXAMINATION

Assessment criteria'S

A 3D rendering of a white puzzle with one red piece in the center. The puzzle pieces are interlocking, and the red piece is the only one of its color, standing out prominently. The lighting creates soft shadows, giving the pieces a three-dimensional appearance.

PROGRAM OUTCOMES MAPPING WITH CO2

DISCUSS AND
UNDERSTAND VARIOUS
COMBINATIONAL LOGIC
CIRCUITS

Combinational Logic

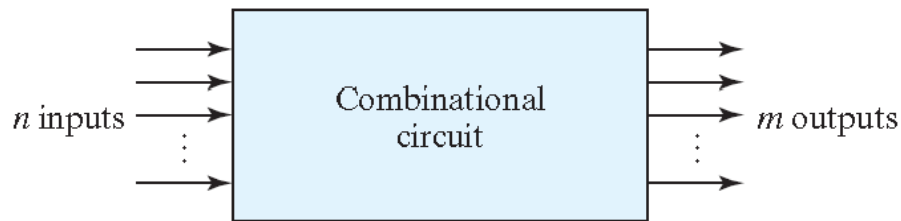
1. Introduction
2. Combinational Circuits
3. Binary Adder–Subtractor
4. Decimal Adder
5. Binary Multiplier
6. Magnitude Comparator
7. Decoders
8. Encoders
9. Multiplexers

Introduction

- Logic circuits may be *combinational* or *sequential*.
- A combinational circuit consists of logic gates whose outputs at any time are determined from only the present combination of inputs.
- The operation of combinational circuits can be specified logically by a set of Boolean functions.
- Sequential circuits contain *storage* elements in addition to logic gates.
- The outputs of sequential circuits are a function of the inputs and the state of the storage elements.
- Because the state of the storage elements is a function of previous inputs, the outputs of a sequential circuit depend not only on present values of inputs, but also on past inputs, and the circuit behavior must be specified by a time sequence of inputs and internal states.
- Sequential circuits are discussed in Chapters 5 and 8 .

Combinational Circuits

- A combinational circuit consists of an interconnection of logic gates.
- Combinational circuits react to the values at their inputs and produce the value of the output signal, transforming binary information from the given input data to a required output data.
- A block diagram of a combinational circuit is shown.



- The n inputs come from an external source; the m outputs are produced by the combinational circuit and go to an external destination.
- Each input and output variable is an analog electrical signal whose values are interpreted to be a binary signal that represents logic 1 and logic 0.

- For n input variables, there are 2^n possible combinations.
 - For each possible input combination, there is one possible value for each output.
 - Thus, a combinational circuit can be specified with a truth table that lists the output values for each combination of input variables.
 - A combinational circuit also can be described by m Boolean functions, one for each output variable.
 - Each output function is expressed in terms of the n input variables.
-
- Several extensively used combinational circuits, such as adders, subtractors, multipliers, comparators, decoders, encoders, and multiplexers, are available in standard integrated circuit components and used as *standard cells* in complex very large scale integrated (VLSI) circuits.

Binary Adder-Subtractor

- **Half adder**

- $0 + 0 = 0$; $0 + 1 = 1$; $1 + 0 = 1$; $1 + 1 = 10$
- two input variables: x, y
- two output variables: C (carry), S (sum)
- truth table

Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

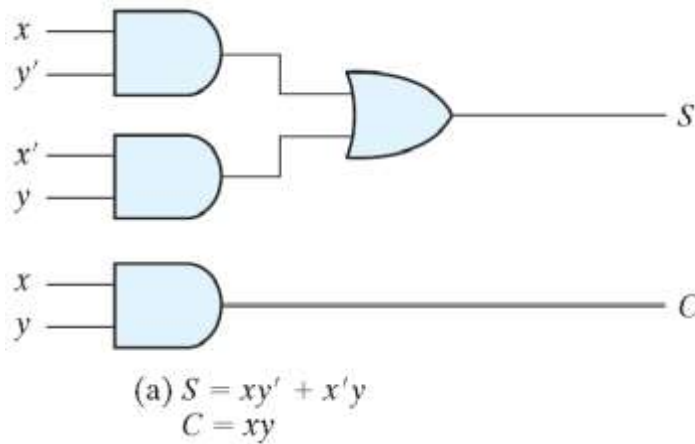
- Boolean functions

$$S = x'y + xy'$$

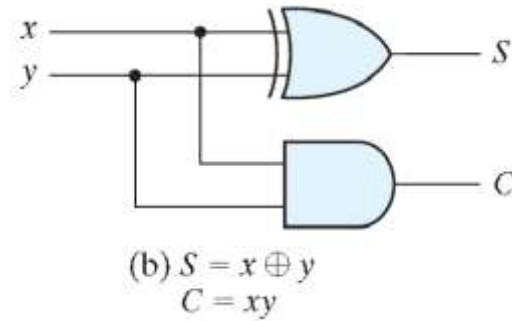
$$C = xy$$

Implementation of Half Adder

- sum of products



exclusive-OR and AND



Full-Adder

- The arithmetic sum of three input bits
- Three input bits
 - x, y : two significant bits
 - z : the carry bit from the previous lower significant bit
- Two output bits: C, S
- Truth table

Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

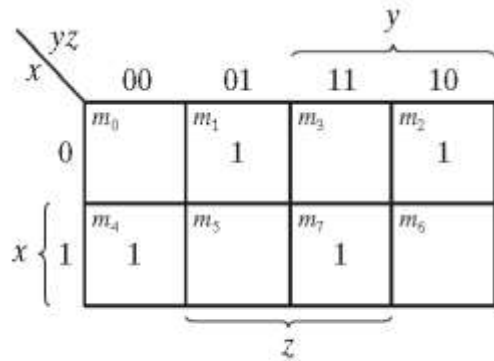
Boolean Functions of Full Adder

$$S = x'y'z + x'yz' + xy'z' + xyz$$

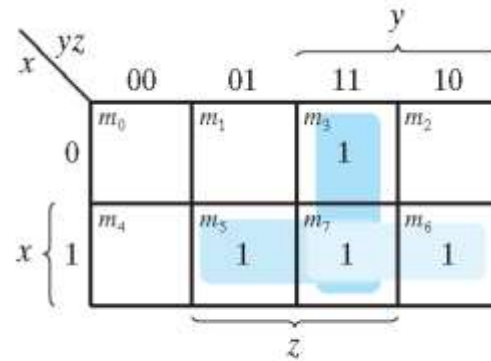
$$C = xy + xz + yz$$

Full Adder

x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



(a) $S = x'y'z + x'yz' + xy'z' + xyz$



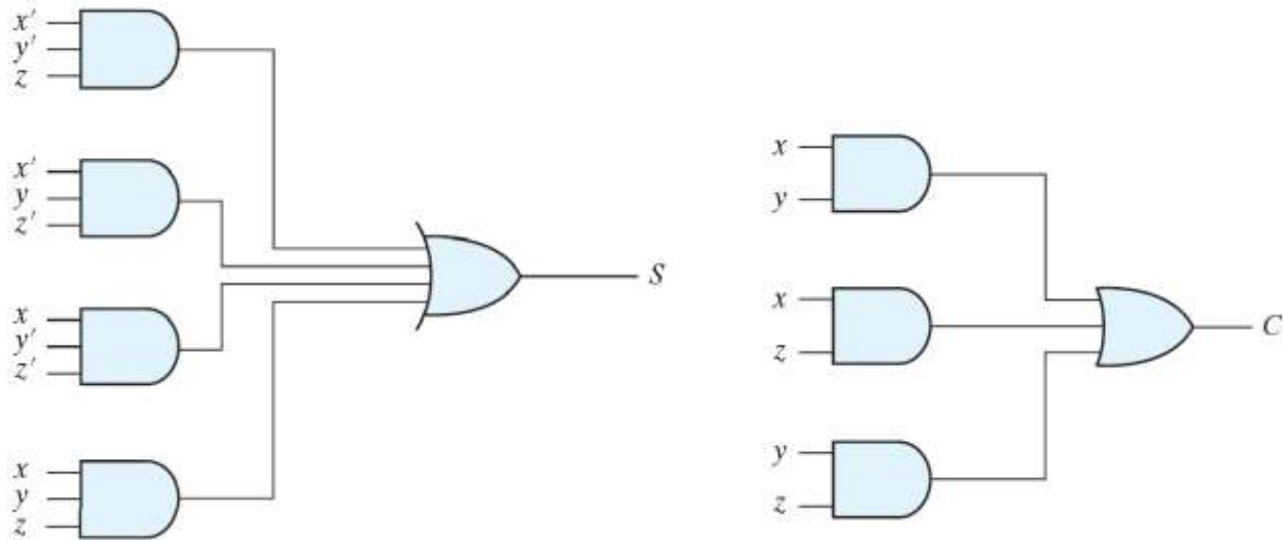
(b) $C = xy + xz + yz$

$$S = x \oplus y \oplus z$$

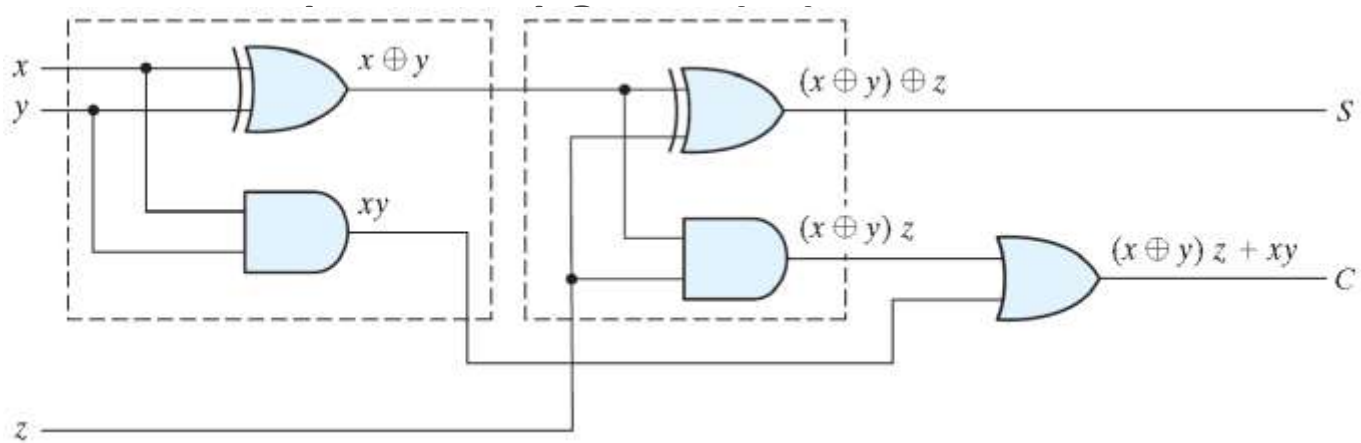
$$C = xy + (x + y)z$$

Sum-of-Product

Implementation of Full Adder



Implementation of Full Adder



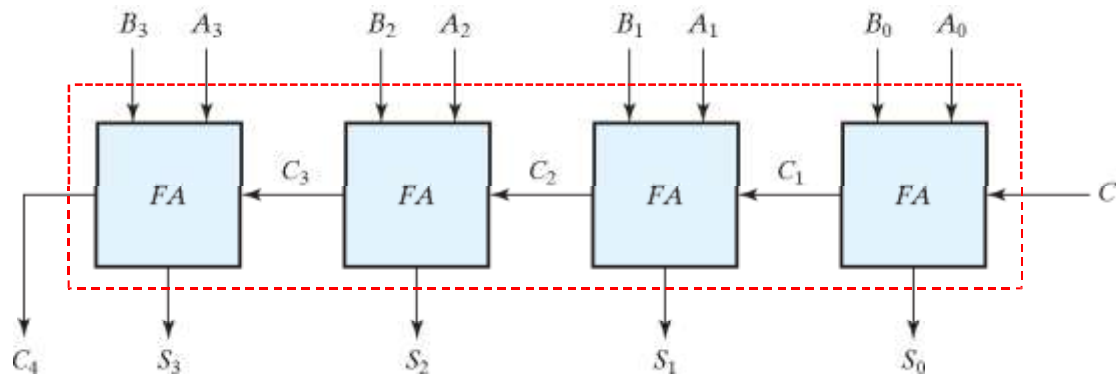
$$S = x \oplus y \oplus z$$

$$\begin{aligned} C &= xy + (x + y)z \\ &= xy + (x \oplus y)z \end{aligned}$$

Binary Adder

- A binary adder can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain (called *ripple-carry* adder).
- Example: 4-bit binary addder

Is it possible to design it with truth table?



$A = 1011$

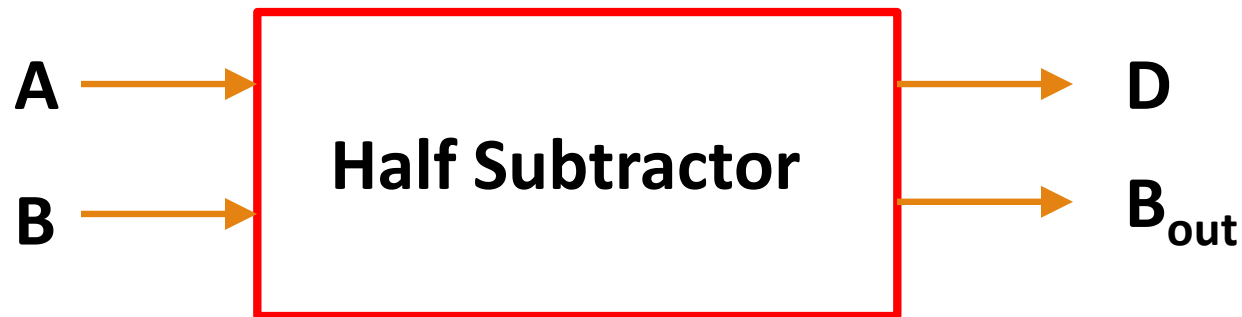
$B = 0011$

$S = 1110$

Subscript i :	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

Binary Half Subtractor

- Subtracting a single-bit binary value B from another A (i.e. $A - B$) produces a difference bit D and a borrow out bit B_{out} .
- This operation is called half subtraction and the circuit to realize it is called a half subtractor.



Truth Table for Half Subtractor

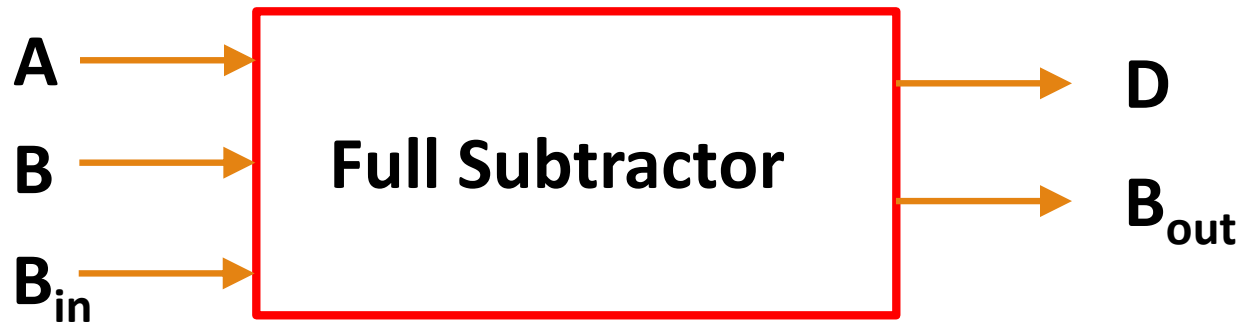
A	B	D	B _{out}
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

D =

B_{out} =

Binary Full Subtractor

- Subtracting two single-bit binary values, B, B-in from a single-bit value A produces a difference bit D and a borrow out B-out bit. This is called full subtraction.



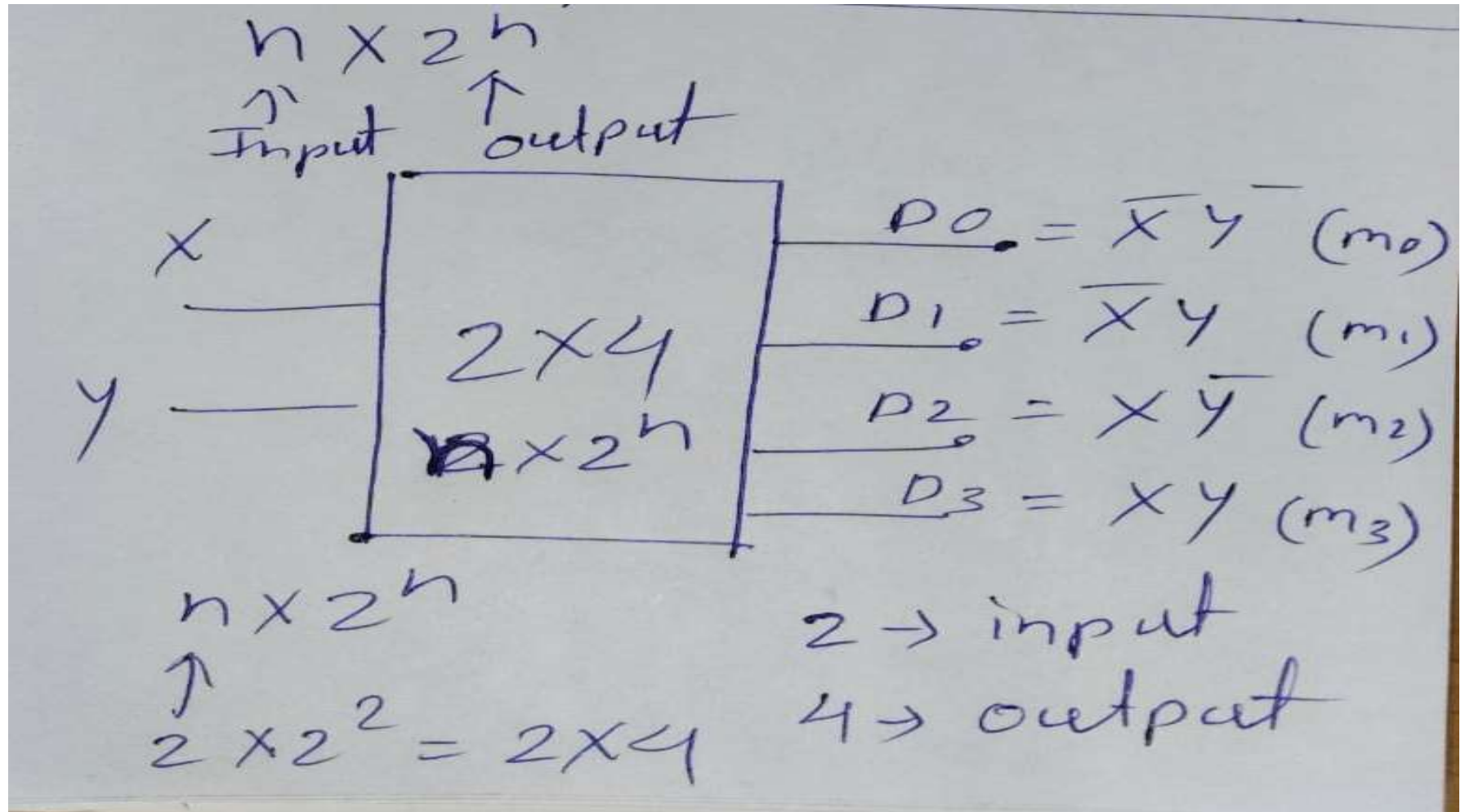
Truth Table for Half Subtractor

A	B	B _{in}	D	B _{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

D =

B_{out} =

DECODER



2x4 En Codes

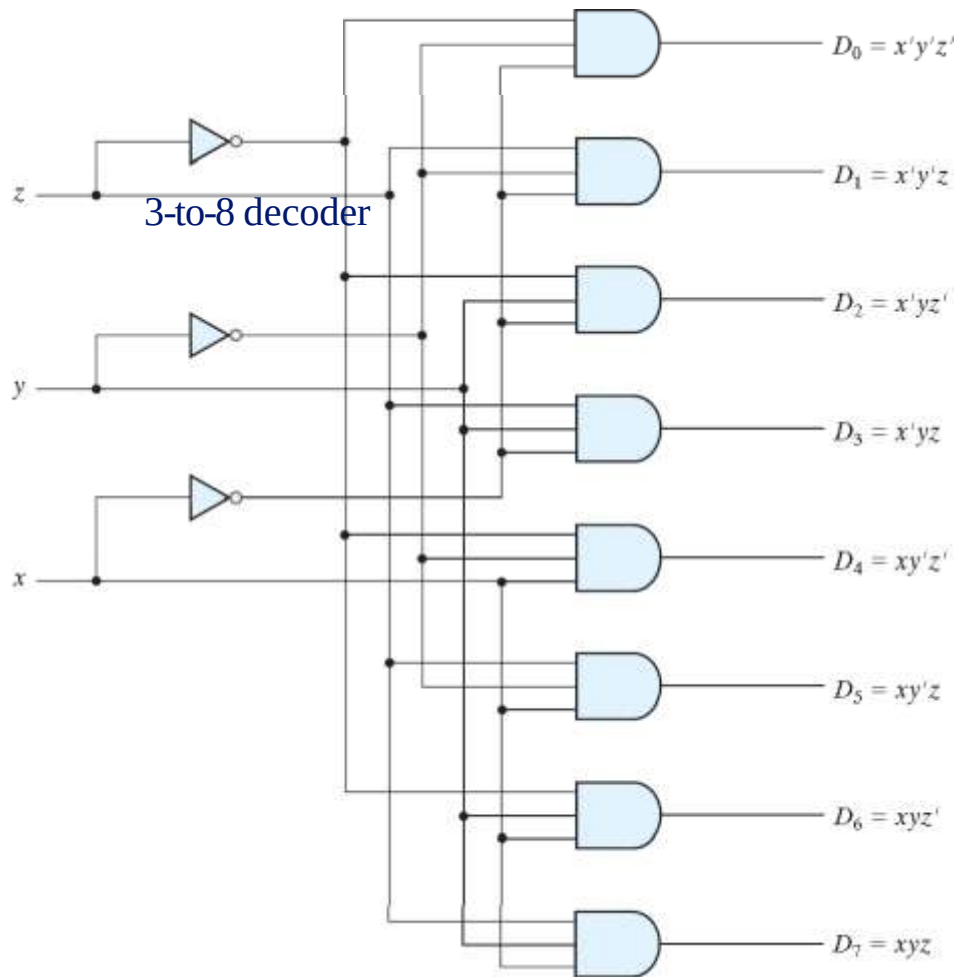
X	Y	D ₀	D ₁	D ₂	D ₃
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Decoders

- A *decoder* converts binary information from n input lines to a maximum of 2^n unique output lines.
- A n -to- m decoder ($m \leq 2^n$)
 - a binary code of n bits has 2^n distinct information
 - n input variables; up to 2^n output lines
 - only one output can be active (high) at any time

Truth Table of a Three-to-Eight-Line Decoder

Inputs			Outputs							
x	y	z	D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1



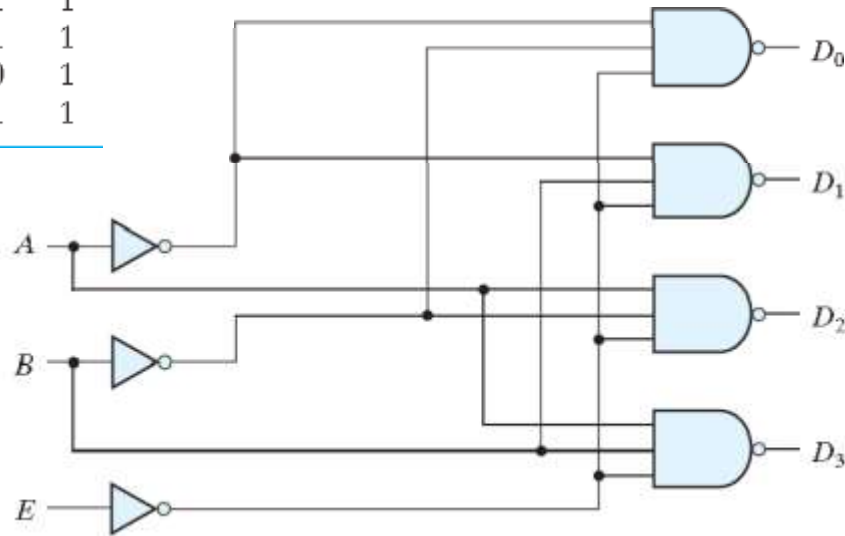
3-to-8 decoder

each output = a minterm

Two-to-four Decoder with Enable

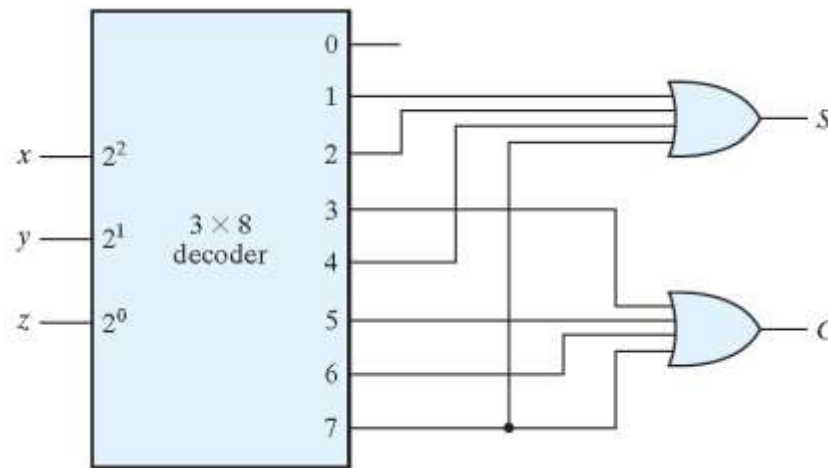
- *Enable* input is added to control the circuit operation.

E	A	B	D_0	D_1	D_2	D_3
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	1



Universal Combinational Logic Implementation

- A decoder provides the 2^n minterms of n input variables.
- A decoder and an external OR gate can implement any Boolean function of n input variables in sum-of-minterm form.
- For example, see Table 4.4, a full-adder has its sum $S(x,y,z) = \Sigma(1,2,4,7)$ and carry $C(x,y,z) = \Sigma(3,5,6,7)$.



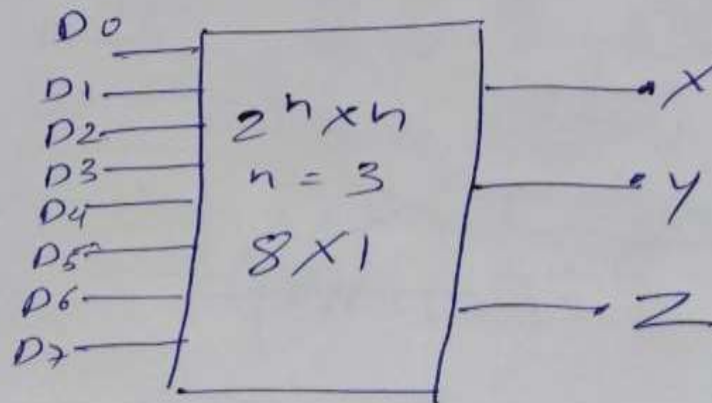
- Two possible approaches using decoder
 - OR(minterms of F): k inputs
 - NOR(minterms of F'): $2^n - k$ inputs

Encoder

↳ Inverse of Decoder

$$2^n \times n$$

↑ ↑
Input output



Encoders

- The inverse function of a decoder
- 2^n (or fewer) input lines and n output lines
- The output lines generate the binary code corresponding to the input value.
- Example:

Truth Table of an Octal-to-Binary Encoder

Inputs								Outputs		
D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$z = D_1 + D_3 + D_5 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$x = D_4 + D_5 + D_6 + D_7$$

can be implemented with OR gates

Priority Encoder

- Encoder that includes the priority function
- Resolve the ambiguity of illegal inputs, only one of the input is encoded, the input having the highest priority will take precedence.
- Example:

Truth Table of a Priority Encoder

Inputs				Outputs		
D_0	D_1	D_2	D_3	x	y	V
0	0	0	0	X	X	0
1	0	0	0	0	0	1
X	1	0	0	0	1	1
X	X	1	0	1	0	1
X	X	X	1	1	1	1

- » D_3 has the highest priority
- » D_0 has the lowest priority
- » X: don't-care conditions
- » V : valid output indicator

Maps for Simplifying x and y

$D_0D_1 \backslash D_2D_3$		D_2			
		00	01	11	10
D_0	00	m_0 X	m_1 1	m_3 1	m_2 1
	01	m_4	m_5 1	m_7 1	m_6 1
	11	m_{12}	m_{13} 1	m_{15} 1	m_{14} 1
	10	m_8	m_9 1	m_{11} 1	m_{10} X

D_1 is indicated by a bracket on the right side of the table, covering rows 01, 11, and 10.

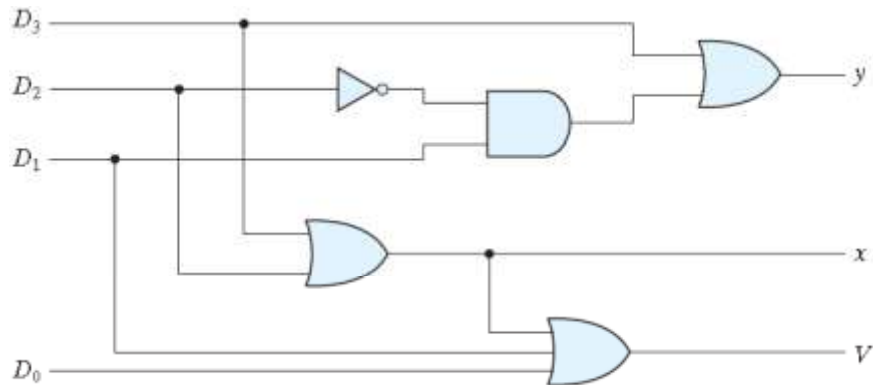
D_3 is indicated by a bracket at the bottom of the table, covering columns 01, 11, and 10.

$D_0D_1 \backslash D_2D_3$		D_2			
		00	01	11	10
D_0	00	m_0 X	m_1 1	m_3 1	m_2
	01	m_4 1	m_5 1	m_7 1	m_6
	11	m_{12} 1	m_{13} 1	m_{15} 1	m_{14}
	10	m_8	m_9 1	m_{11} 1	m_{10}

D_1 is indicated by a bracket on the right side of the table, covering rows 01, 11, and 10.

D_3 is indicated by a bracket at the bottom of the table, covering columns 01, 11, and 10.

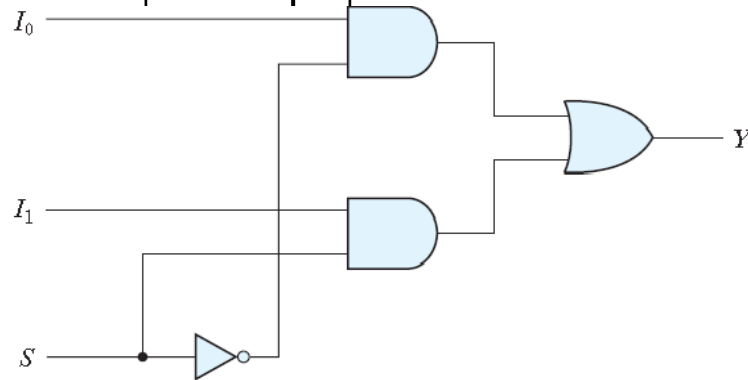
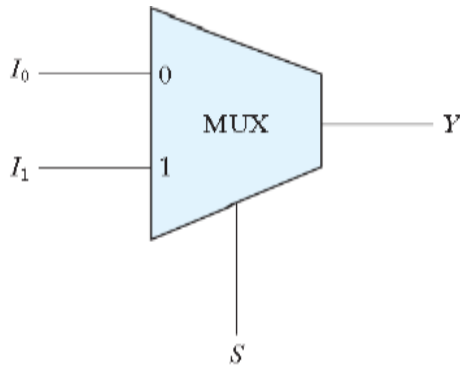
- $x = D_2 + D_3$
- $y = D_3 + D_1 D_2'$
- $V = D_0 + D_1 + D_2 + D_3$



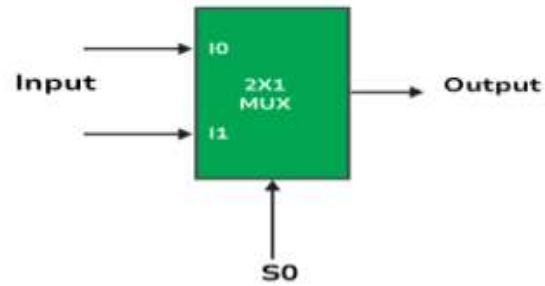
Multiplexers

- Select from one of many inputs and directs it to a single output, controlled by a set of selection lines. A multiplexer is also called a *data selector*.
- Normally, there are 2^n inputs and n selection lines whose bit combinations determine which input is selected.
- Example: (two-to-one multiplexer)
 - $Y = I_0$ if $S = 0$, and $Y = I_1$ if $S = 1$.
 - $Y = S'I_0 + SY_1$

S	I_0	I_1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1



2:1 Multiplexer

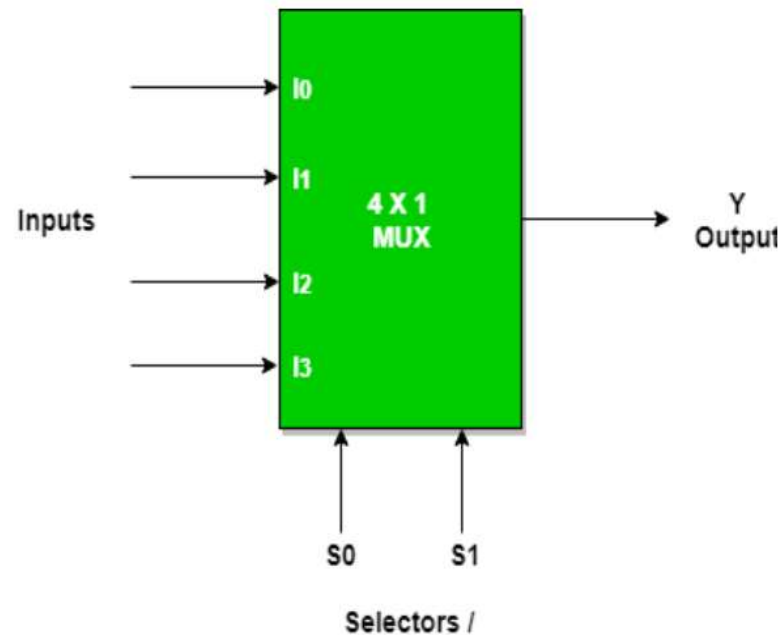


Truth Table

S_0	I_0	I_1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

Block Diagram of 4×1 Multiplexer

In the Given Block Diagram I0, I1, I2, and I3 are the 4 inputs and Y is the Single output which is based on Select lines S0 and S1.



The output of the multiplexer is determined by the binary value of the selection lines

- When $S_1S_0=00$, the input I_0 is selected.
- When $S_1S_0=01$, the input I_1 is selected.
- When $S_1S_0=10$, the input I_2 is selected.
- When $S_1S_0=11$, the input I_3 is selected.

Truth Table of 4×1 Multiplexer

Given Below is the Truth Table of 4x1 Multiplexer

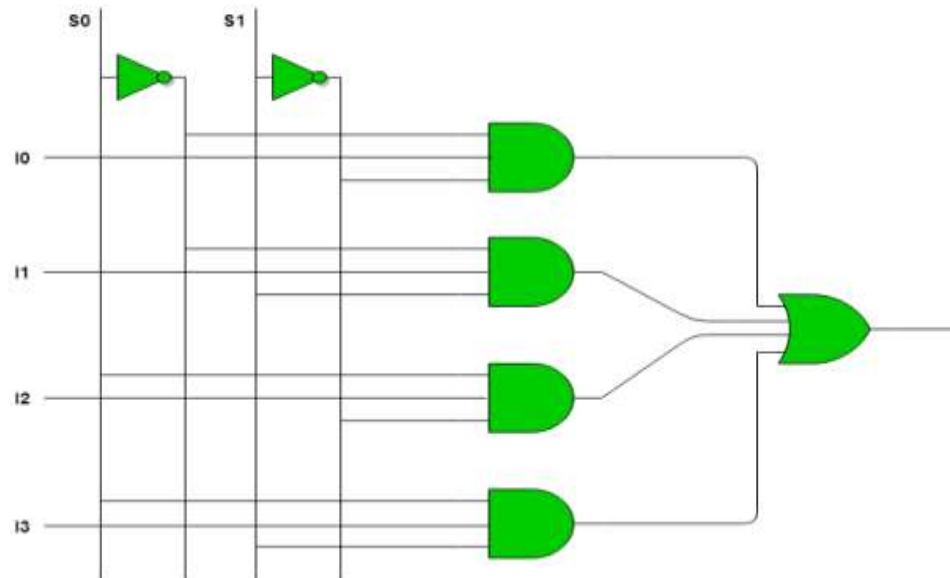
Truth Table

S_0	S_1	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Y = S_0'.S_1'.I_0 + S_0'.S_1.I_1 + S_0.S_1'.I_2 + S_0.S_1.I_3$$

Circuit Diagram of 4x1 Multiplexers

Using truth table the circuit diagram can be given as

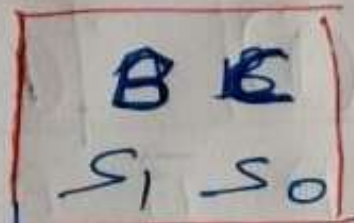




Implementation of Boolean Expression by MUX (multiplexer) } Three variable

Steps: ① Draw the K map of boolean
expression

② Assign the following to select
lines



$B = S_1$, $C = S_0$

③ Write the I_0, I_1, I_2 and I_3
according to value of $BC (S_1 S_0)$





- ④ write the variable A according to selection in the following way:
- (a) if A and $\bar{A}$ both 1, write '1'
 - (b) if only $\bar{A}$ is 1, write ' $\bar{A}$ '
 - (c) if only A is 1, write 'A'
 - (d) if both A and $\bar{A}$ are 0, write '0'

- ⑤ draw the ~~or~~ MUX as B and C connect to s_1, s_0 and mention I_0, I_1, I_2, I_3 for value of A, $\bar{A}$, 0 or 1

Implement Full Adder * No need to solve K-map

$$\text{Sum} = \sum m(1, 2, 4, 7)$$

Step 1 (K-map)

A \ BC	00	01	11	10
	$\bar{B}\bar{C}$	$\bar{B}C$	BC	$B\bar{C}$
$\bar{A}$		1		1
A	1		1	
	A	$\bar{A}$	A	$\bar{A}$

Step 2

BC
S₁ S₀

$$B = S_1, C = S_0$$

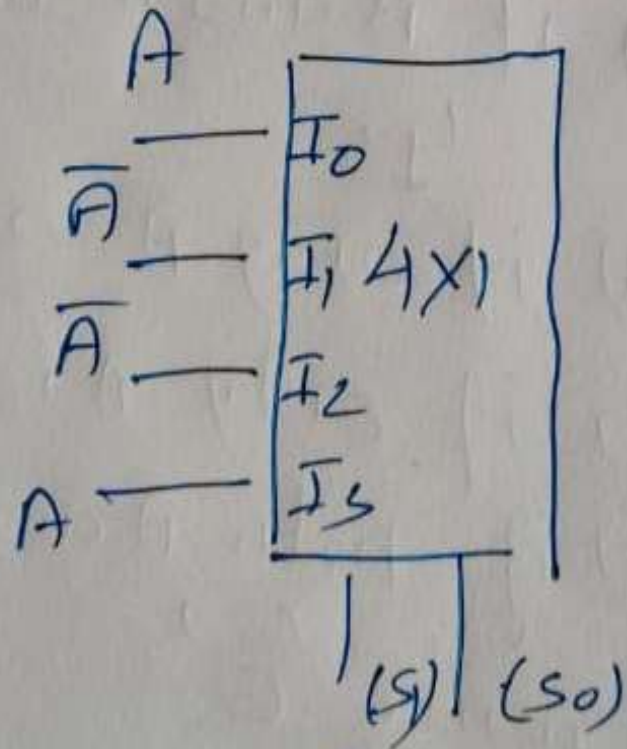
Step 3
84

$I_0 \quad I_1 \quad I_3 \quad I_2$

$$I_0 = A, I_1 = \bar{A}, I_2 = \bar{A}$$

$$I_3 = A$$

step 5



Q Implement carry of Full Adder

$$\text{Carry} = \sum m(3, 5, 6, 7)$$

	A	B	C	
		0	0	0
		0	1	1
		1	1	1
		1	0	0
$\overline{A}$		$\overline{B}\overline{C}$	$\overline{B}C$	$B\overline{C}$
A		$\overline{B}\overline{C}$	$\overline{B}C$	$B\overline{C}$
	0	A	1	A

MUX Table		
S_1	S_0	output
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

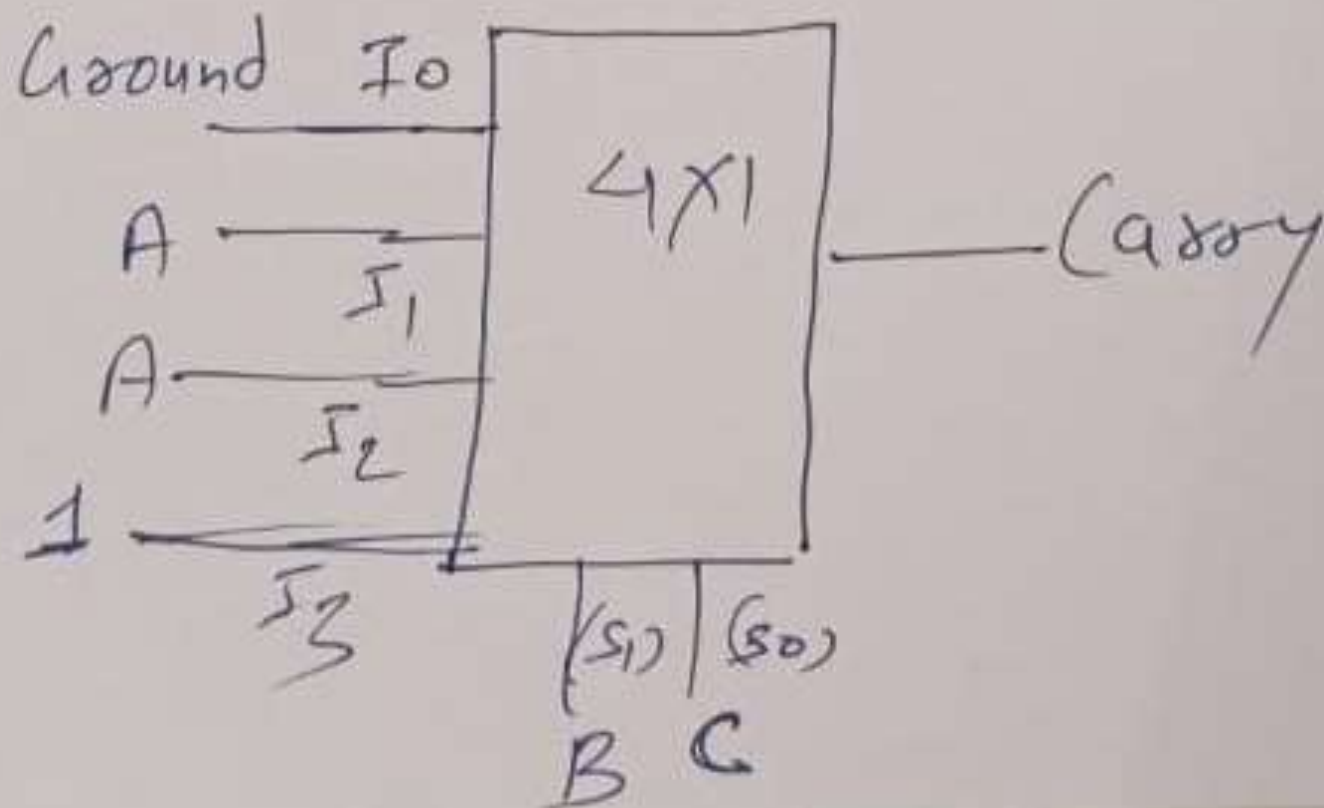
$I_0 \quad I_1 \quad I_3 \quad I_2$

$$\rightarrow I_0 = 0, I_1 = A, I_2 = A$$

$$I_3 = 1$$



$$B = S_1, C = S_0$$

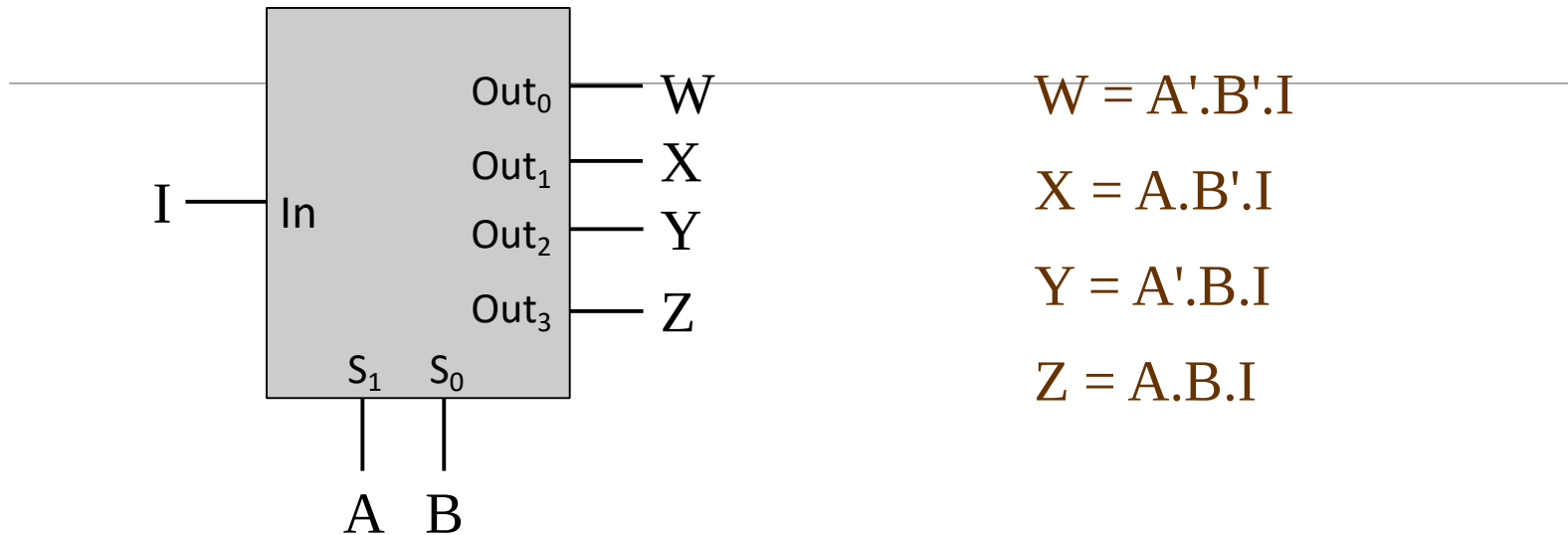


Demultiplexers

- A demultiplexer has

 - N control inputs
 - 1 data input
 - 2^N outputs
- A demultiplexer routes (or connects) the data input to the selected output.
 - The value of the control inputs determines the output that is selected.
- A demultiplexer performs the opposite function of a multiplexer.

Demultiplexers



A	B	W	X	Y	Z
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

Code Conversion

Example

- BCD to excess-3 code
 - The truth table

Table 4.2
Truth Table for Code Conversion Example

Input BCD				Output Excess-3 Code			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>w</i>	<i>x</i>	<i>y</i>	<i>z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

AB \ CD		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3	m_2 1
	01	m_4 1	m_5	m_7	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9	m_{11} X	m_{10} X

$$z = D'$$

AB \ CD		C			
		00	01	11	10
A	00	m_0 1	m_1	m_3 1	m_2
	01	m_4 1	m_5	m_7 1	m_6
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9	m_{11} X	m_{10} X

$$y = CD + C'D'$$

AB \ CD		C			
		00	01	11	10
A	00	m_0	m_1 1	m_3 1	m_2 1
	01	m_4 1	m_5	m_7	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8	m_9 1	m_{11} X	m_{10} X

$$x = B'C + B'D + BC'D'$$

AB \ CD		C			
		00	01	11	10
A	00	m_0	m_1	m_3	m_2
	01	m_4	m_5 1	m_7 1	m_6 1
	11	m_{12} X	m_{13} X	m_{15} X	m_{14} X
	10	m_8 1	m_9 1	m_{11} X	m_{10} X

$$w = A + BC + BD$$

Simplified Functions and Logic Diagram

$$z = D'$$

$$y = CD + C'D' = CD + (C + D)'$$

$$\begin{aligned} x &= B'C + B'D + BC'D' = B'(C + D) + BC'D' \\ &= B'(C + D) + B(C + D)' \end{aligned}$$

$$w = A + BC + BD = A + B(C + D)$$

